TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PASHCHIMANCHAL CAMPUS

REDSENTINEL: TRANSFORMER-BASED GENERATION AND OBFUSCATION OF CONTEXT-AWARE XSS PAYLOADS

BY:

BIPIN BHATTRAI	**PAS078BCT009**
SEAMOON PANDEY	**PAS078BCT035**
SWORUP BHANDARI	**PAS078BCT046**

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE BACHELOR'S DEGREE IN COMPUTER ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

POKHARA, NEPAL

May, 2026

LETTER OF APPROVAL

The undersigned certify that they have read and recommended to the Institute of Engineering for acceptance, a project report entitled **'RedSentinel: Transformer-Based Generation and Obfuscation of Context-Aware XSS Payloads'** submitted by **Bipin Bhattrai, Seamoon Pandey and Sworup Bhandari** in partial fulfillment of the requirements for the Bachelor's Degree in Computer Engineering.

Supervisor, **Asst. Prof. Sudip Dahal**
Department of Electronics and Computer Engineering
Institute of Engineering, Paschimanchal Campus

External Examiner, **Bidur Devkota, PhD**
Assistant Professor, Gandaki College of Engineering and Sciences
Gandaki province, Kaski

Coordinator, **Asst. Prof. Hom Nath Tiwari**
Head of Department
Department of Electronics and Computer Engineering
Institute of Engineering, Paschimanchal Campus

DATE OF APPROVAL : 25 MAY 2026

COPYRIGHT

ACADEMIC AND ETHICAL DECLARATION

This report is written for academic analysis and defensive software engineering. The project domain is dual-use: an XSS scanner can support authorized vulnerability assessment, but the same technical knowledge may be misused against systems without permission. Accordingly, the report avoids operational instructions for unauthorized exploitation and frames payload generation,fuzzing, and browser verification as controlled defensive testing capabilities. Any implementation ordemonstration should be limited to owned systems, intentionally vulnerable labs, bug-bounty scopes, or environments where explicit written authorization exists. The evaluation in this report is based on repository evidence available through connected source inspection and selected external academic/security references. Repository evaluation claims are treated as project-provided evidence unless independently executed. The report therefore distinguishes between implemented code, documented behavior, and evaluation assertions.

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to everyone who contributed to the successful completion of this report and the development of the Red Sentinel project. First and foremost, we extend our deepest appreciation to our project supervisor, **Mr. Sudip Dahal**, for his invaluable guidance, continuous support, and constructive feedback throughout this research. His expertise in web security, machine learning, and computer networking has been instrumental in shaping the direction and quality of our work. We would also like to sincerely thank our respected teachers, Mr. Hari Prasad Baral and Mr. Hom Nath Tiwari, for their encouragement, valuable suggestions, and academic support during the course of this project.

We are also grateful to our friends and colleagues Samar Dotel, Aman CK, Anish Koirala, Shweata Koirala, Shristi Panthi and Rakhsya Raut for their constant motivation, cooperation, and support throughout the completion of this project. Finally, we would like to thank the Department of Electronics and Computer Engineering at Tribhuvan University Pashchimanchal Campus for providing the essential resources, infrastructure, and academic environment that made this project possible.

Bipin Bhattarai
Seamoon Pandey
Sworup Bhandari

May, 2026

ABSTRACT

Cross-Site Scripting (XSS) remains a persistent web vulnerability and traditional scanners, dependent on signatures and manually crafted payloads, struggle with modern filtering mechanisms and diverse injection contexts. The system employs a polyglot microservice architecture combining a TypeScript/NestJS orchestration core with three specialized Python-based AI microservices, all coordinated through a distributed job queue backed by Redis and surfaced via a real-time Next.js dashboard.

The core innovation lies in the integration of a fine-tuned DistilBERT transformer model for injection context classification, combined with a 59,122-entry labeled XSS payload dataset and a multistrategy payload mutation and obfuscation engine. This allows RedSentinel to reason about the semantic context of injection points, distinguishing HTML attribute contexts from JavaScript sink contexts, DOM-based sinks from server-reflected contexts, and generate payloads that are specifically crafted to survive filtering within that context.

The scanner performs an end-to-end pipeline consisting of optional target authentication, crawling, context analysis, payload generation, fuzzing, and report generation. Payload generation uses a curated payload bank of approximately 59,122 entries, mutation and obfuscation strategies, and XGBoost ranking when the trained ranker artifact is available, with heuristic fallback otherwise. The fuzzer checks reflection, supports DOM-oriented and stored-XSS paths with limitations, and can use headless browser verification to distinguish executable payloads from plain textual reflection

This report provides a comprehensive technical account of the system's architecture,module design, API contracts, dataset and model artifacts, scan pipeline, evaluation results and testing approach of RedSentinel.

Keywords: Cross-Site Scripting, XSS, Machine Learning, Transformer Models, Adversarial Payload Generation, Web Security, Automated Penetration Testing, Obfuscation, Context-Aware Attack Generation, BullMQ, Uvicorn, Pydantic, Playwright

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

List of Abbreviations

AI Artificial Intelligence

API Application Programming Interface

AWS Amazon Web Services

CI/CD Continuous Integration/Continuous Deployment

CVE Common Vulnerabilities and Exposures

DDQN Double Deep Q-Network

DOM Document Object Model

DQN Deep Q-Network

DVWA Damn Vulnerable Web Application

GRU Gated Recurrent Unit

HTML HyperText Markup Language

LSTM Long Short-Term Memory

ML Machine Learning

mTLS Mutual Transport Layer Security

NLP Natural Language Processing

OWASP Open Web Application Security Project

RL Reinforcement Learning

RNN Recurrent Neural Network

WAF Web Application Firewall

XSS Cross-Site Scripting

CHAPTER 1
INTRODUCTION

1.1 Background

RedSentinel is an AI-augmented web application security scanner purpose-built to detect Cross-Site Scripting (XSS) vulnerabilities. XSS remains one of the most persistently exploited vulnerability classes in modern web applications. According to the OWASP Top 10, injection-related vulnerabilities have appeared consistently across multiple editions of the industry-standard web application risk rankings, with Cross-Site Scripting discussed within A03:2021 (Injection) as a prominent client-side security concern [1]. The widespread prevalence of XSS is further corroborated by the Common Vulnerabilities and Exposures (CVE) database, where XSS routinely accounts for a substantial share of annually reported web vulnerabilities [2].

Traditional XSS scanners operate on static, manually curated payload lists paired with simple string-matching heuristics. While adequate for detecting trivial reflections, these approaches fail in two important respects: they are blind to injection context meaning they cannot distinguish between a parameter reflected inside a JavaScript string literal and one reflected inside an HTML attribute value, contexts that demand entirely different payload structures and they are easily defeated by modern Web Application Firewalls (WAFs), which signature-match against known payload patterns [3].

RedSentinel addresses both limitations through a five-phase pipeline that integrates machine learning-based injection context classification at the core of its analysis workflow. The context classification is performed by a fine-tuned DistilBERT model, a distilled variant of the BERT transformer architecture, introduced by Devlin et al. [4] and subsequently compressed by Sanh et al. [5] to approximately 40% of BERT's parameter count while retaining 97% of its language understanding capability on the GLUE benchmark. The model is fine-tuned on a domain-specific dataset of 59,122 labeled XSS payloads, enabling it to classify injection contexts (HTML body, HTML attribute, JavaScript string, JavaScript code block, URL context, CSS context, and DOM sink) from source code fragments surrounding the injection point.

The system's architecture follows a polyglot microservices pattern. A NestJS/TypeScript orchestration core [6] manages the HTTP API, WebSocket real-time event streaming via Socket.io [7], job queue scheduling via BullMQ backed by Redis [8], and PostgreSQL persistence via TypeORM [9]. Three independently deployable Python

FastAPI microservices [10] handle the AI-intensive concerns: context analysis (port 5001), payload generation with mutation and obfuscation (port 5002), and HTTP/headless-browser fuzzing via Playwright [11] (port 5003). Asynchronous job execution through BullMQ ensures the REST API remains non-blocking while scans execute in background workers, and Socket.io WebSocket events deliver real-time progress and vulnerability findings to the Next.js dashboard [12].

The payload generation subsystem draws on a curated bank of 59,122 labeled XSS payloads. Payload selection is context-aware and WAF-evasion-weighted, followed by a mutation engine that applies systematic transformations (case variation, Unicode and HTML entity encoding, whitespace manipulation, and event-handler alternation) and an obfuscation layer that applies higher-order JavaScript transforms such as Base64/atob encoding, String.fromCharCode construction, and eval indirection. This approach is consistent with established research on WAF bypass techniques, which demonstrates that encoding diversity and structural variation significantly reduce detection rates of signature-based filters [3].

Vulnerability confirmation in the fuzzing phase employs a two-stage verification strategy: HTTP response reflection analysis followed by headless Chromium execution via Playwright, using a canary window property injection technique to confirm actual JavaScript execution rather than mere textual reflection. This dual-verification approach is aligned with best practices in automated XSS confirmation research, which distinguishes passive reflection (a necessary but insufficient condition for XSS) from confirmed execution. The system concludes each scan by generating vulnerability reports in HTML, PDF (via Puppeteer [13]), and JSON formats, suitable for both human review and CI/CD pipeline integration.

1.2 Problem Statement

Although XSS scanners and WAFs can identify many basic vulnerabilities, they often depend on static payloads and basic reflection checks, making them less effective at handling different injection contexts, WAF/filter bypasses, browser execution verification, and structured reporting.

1.3 Objectives

The primary objectives of the RedSentinel are:

1. To design and implement RedSentinel, an AI-assisted, context-aware XSS scanner that crawls targets, analyses injection contexts, generates and ranks payloads, fuzzes and verifies vulnerabilities.
2. To generate structured, actionable vulnerability reports in multiple formats.

CHAPTER 2
LITERATURE REVIEW

2.1 Related Works

The following review examines significant advancements in automated web security testing, with a particular focus on XSS payload generation and adversarial input synthesis. Existing research covers a wide spectrum of techniques, including reinforcement learning, neural machine translation, generative adversarial models, and transformer-based language approaches. By analysing these contributions, this review highlights the methodological trends, strengths, and limitations that shape for the design and development of the Red Sentinel system..

Foley and Maffeis (2022) propose HAXSS, a hierarchical reinforcement-learning framework to automatically generate XSS attack payloads. Their approach frames the payload creation as two nested RL "games." The first agent learns to escape the immediate HTML context of user output (escaping tags or attributes), while the second agent intervenes whenever the application attempts sanitization, it learns to obfuscate the payload to bypass filters. Successful obfuscations are fed back into the first agent for further refinement. Implemented as an end-to-end black-box fuzzer, HAXSS was evaluated on both synthetic benchmarks and real web applications. It identified 131 XSS vulnerabilities (20% more than state-of-the-art scanners) with zero false positives, including rediscovering known CVEs and uncovering 5 novel CVEs in production-grade sites. Thus, HAXSS demonstrates that hierarchical RL can produce diverse, context- and filter-aware payloads that significantly improve XSS discovery over traditional scanners [14].

Khan (2024) presents LL-XSS, an end-to-end deep generative model for crafting XSS payloads. Unlike hand-crafted or brute-fuzzed payloads, LL-XSS leverages a combination of auto-regressive neural networks and transformer architectures to analyze both frontend and backend web code and produce candidate attack scripts. The model is trained on example web application code to learn common injection patterns, then generates new malicious scripts aimed at identified vulnerabilities. In evaluation on the OWASP Juice Shop, the author reports that LL-XSS can automatically generate syntactically valid and exploitable XSS payloads by understanding the target's code context. This approach effectively automates the penetration-testing process: LL-XSS bypasses known filters and exposes vulnerabilities with minimal human guidance, demonstrating the promise of AI-driven payload creation for web security [15].

Frempong et al. (2021) developed HIJaX, a neural machine translation model that converts natural-language attack descriptions into working XSS payloads. The system was trained on paired examples of intent phrases and JavaScript exploits. Their experiments showed that the model could reliably generate valid payloads that triggered real XSS vulnerabilities, demonstrating that neural translation methods can automate exploit creation even for users with limited technical expertise [16].

Pala et al. (2023) examined contemporary XSS scanners and highlighted XSStrike as a tool that combines intelligent fuzzing with basic machine-learning techniques. XSStrike analyzes the structure of a web page to detect injection points and then mutates payloads based on contextual feedback. Their evaluation showed that this approach improves detection and execution of XSS payloads across different contexts, illustrating the value of context-aware fuzzing [17].

Fink (2018) introduced FOXSS, a scanner that integrates static data-flow analysis with targeted, context-sensitive fuzzing. The tool identifies potential input flows and generates tailored payloads for each sink, verifying results in a real browser. FOXSS demonstrated very high detection accuracy, outperforming conventional scanners and significantly reducing false positives, indicating that program analysis combined with adaptive fuzzing can greatly enhance XSS detection [18].

Song et al. (2023) presented a grey-box fuzzing system that uses reinforcement learning to create and refine XSS payloads. After mapping all input points through static analysis, the authors trained RL agents (DQN, DDQN, and Policy Gradient) to adjust payloads based on execution feedback. The RL-based fuzzer identified all known XSS vulnerabilities in benchmark applications with no false positives, demonstrating that RL can effectively adapt payloads to complex contexts [19].

Miczek et al. (2025) explored the use of large language models to generate obfuscated XSS attacks capable of evading machine-learning detectors. Their study showed that classifiers trained on standard payloads performed poorly when encountering obfuscated variants. By fine-tuning a transformer to generate diverse obfuscations, the authors significantly improved detector robustness after retraining. This work highlights the usefulness of LLM-generated adversarial examples for strengthening defensive models [20].

CHAPTER 3
METHODOLOGY

3.1 Architectural Design Overview

Red Sentinel is implemented using a microservice architecture. In this design, each major functional component of the system runs as an independent service. These services communicate via well-defined APIs and are coordinated by a central orchestration core the "Core Module." This microservice approach ensures that each module is isolated, independently deployable, testable, and can be scaled or replaced without impacting the rest of the system addressing concerns of maintainability, modularity, and system complexity typical in security tools.

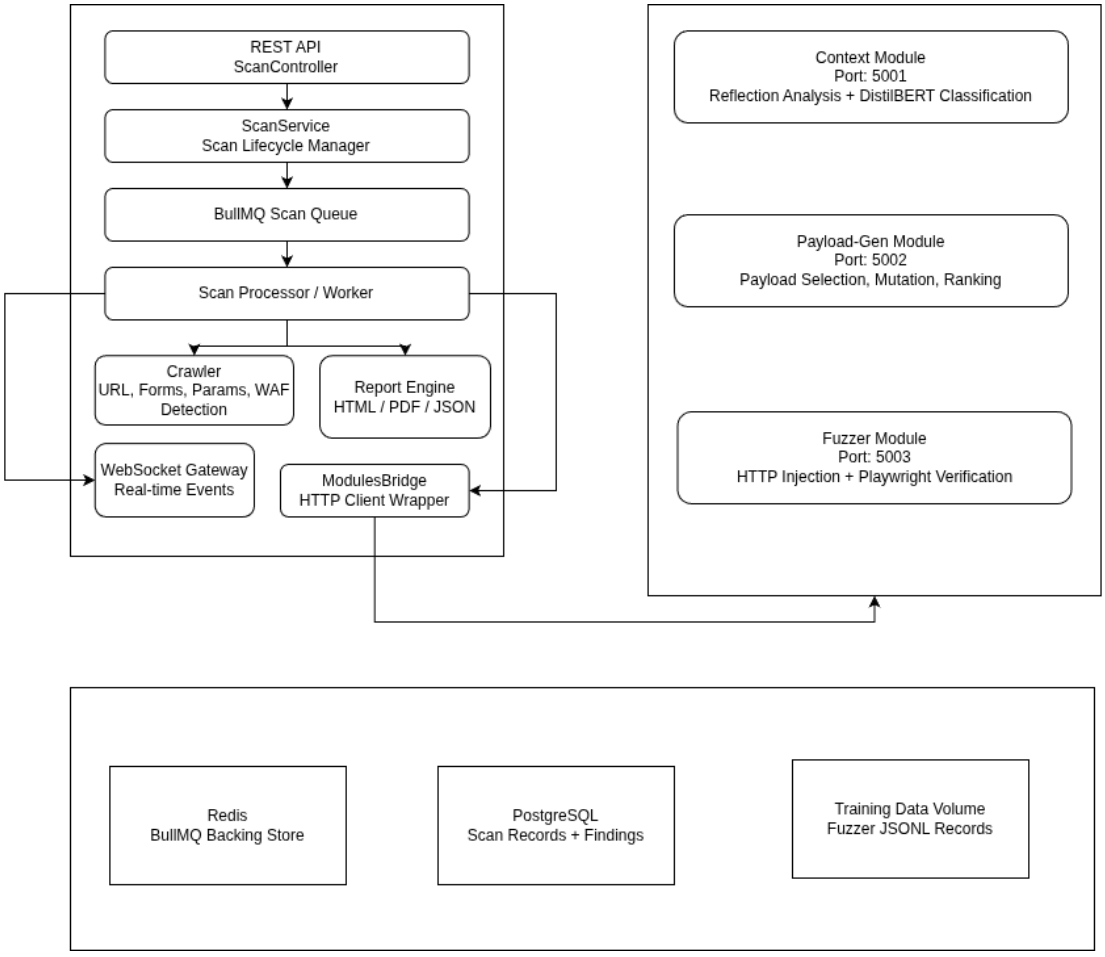

Figure 3.1: *System Architecture Diagram*

The architecture consists of six primary services: a NestJS core service providing the REST API, WebSocket gateway, and job queue processor; three Python FastAPI mi-

croservices (Context, Payload-Gen, Fuzzer); a Next.js dashboard; and two infrastructure services (Redis and PostgreSQL).

3.1.1 Design Principles

3.1.1.1 Separation of Concerns

Each microservice is responsible for a single phase of the scanning pipeline. The Context module understands the injection context, the Payload Generation module understands payload selection and mutation, the Fuzzifier module understands HTTP-level injection and browser verification. The Core co-ordinates these phases without containing their implementation logic. This separation allows each service to be modified independently and be tested in isolation.

3.1.1.2 Asynchronous Job Processing

Scan jobs are not processed synchronously in the HTTP request-response cycle. When a client submits a POST /scan request, the Core immediately persists the scan record to PostgreSQL, enqueues a BullMQ job, and returns the scan ID to the client. The actual scanning pipeline executes asynchronously in a BullMQ worker process. This design decouples request handling from compute intensive scanning, allows multiple scans to be queued without blocking, and supports scan cancellation, retries, and priority management.

3.1.1.3 Event-Driven Real-Time Updates

The BullMQ scan processor emits events at each pipeline phase transition and upon each vulnerability discovery. These events are relayed to the NestJS WebSocket Gateway, which broadcasts them to subscribed Socket.io clients. This event-driven architecture provides clients with sub-second latency updates without polling.

3.1.1.4 Health-Check Dependency Management

Docker Compose service startup is conditioned on health checks: the Core service will not start until Redis, PostgreSQL, and all three Python microservices report healthy. Each Python service exposes a /health endpoint returning service status and model load state. This prevents race conditions during cold starts where the Core might attempt to call Python services before their AI models have loaded.

3.1.2 Data Flow in Scan Execution

A complete scan execution follows this data flow:

1. Client submits POST /scan with target URL, scan depth, and options.
2. ScanController validates request, persists ScanRecord to PostgreSQL with status PENDING, enqueues BullMQ job with scan ID.
3. BullMQ Processor picks up job, updates status to RUNNING, begins Phase 1: CRAWL.

4. Crawler spiders target URL up to configured depth, collecting all discovered URLs, query parameters, form fields, headers, and cookies. WAF detection heuristics run concurrently. Example:

```
Dicovered page:
 <script>
     var searchTerm = "USER_INPUT";
   </script>

            Crawler output:
            {
       "url": "https://demo-app.com/search?q=test",
       "parameter": "q",
       "reflectionPoint": "script_tag"}
```

5. Crawler results are passed to ModulesBridge, which issues POST /analyze to Context module (:5001).
6. Context module probes each injection point with probe strings, analyzes reflections, and classifies each injection point's context using DistilBERT.
7. Context classification results are passed to ModulesBridge, which issues POST /generate to Payload-Gen module (:5002).
8. Payload-Gen module selects candidates from the dataset filtered by context, applies mutation and obfuscation, ranks payloads using a secondary ranker model. Example:

```
{
   {    "payloads": [
         "\";alert(1);//",
         "\"};alert(document.domain);//",
        ..........
       ]
     }
```

9. Ranked payloads are passed to ModulesBridge, which issues POST /fuzz to Fuzzer module (:5003).
10. Fuzzer injects payloads, checks HTTP responses for reflection, launches Playwright headless browser for verification of apparent reflections. Confirmed executions are reported as vulnerabilities. Example:

```
{
   Injected response:
         <script>
  var searchTerm = "";alert(1);//";
</script>
      }
```

11. Report Engine generates HTML/PDF/JSON reports. Scan status is set to COMPLETED. scan:complete WebSocket event is broadcast.

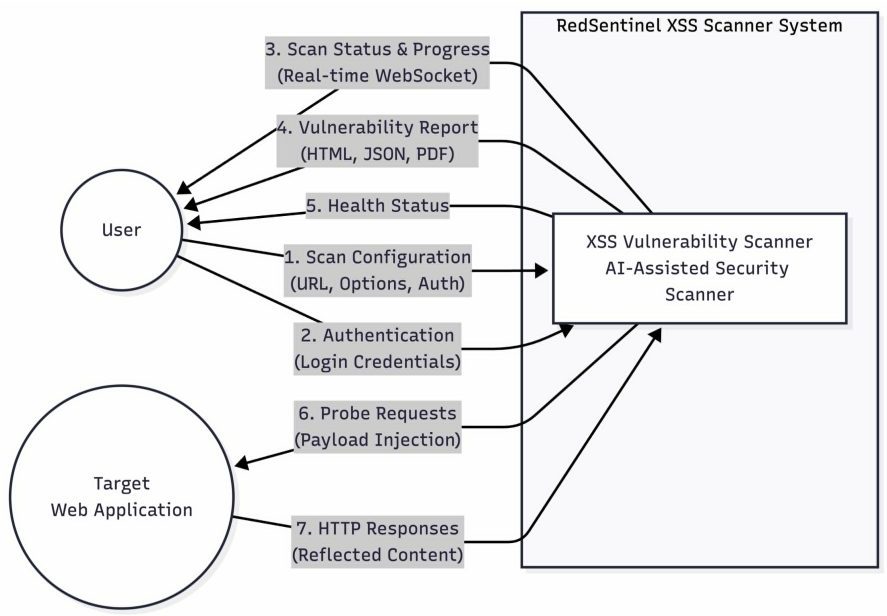

Figure 3.2: *High-Level Architecture and Communication Flow of the RedSentinel AI-Assisted XSS Vulnerability Scanner System.*

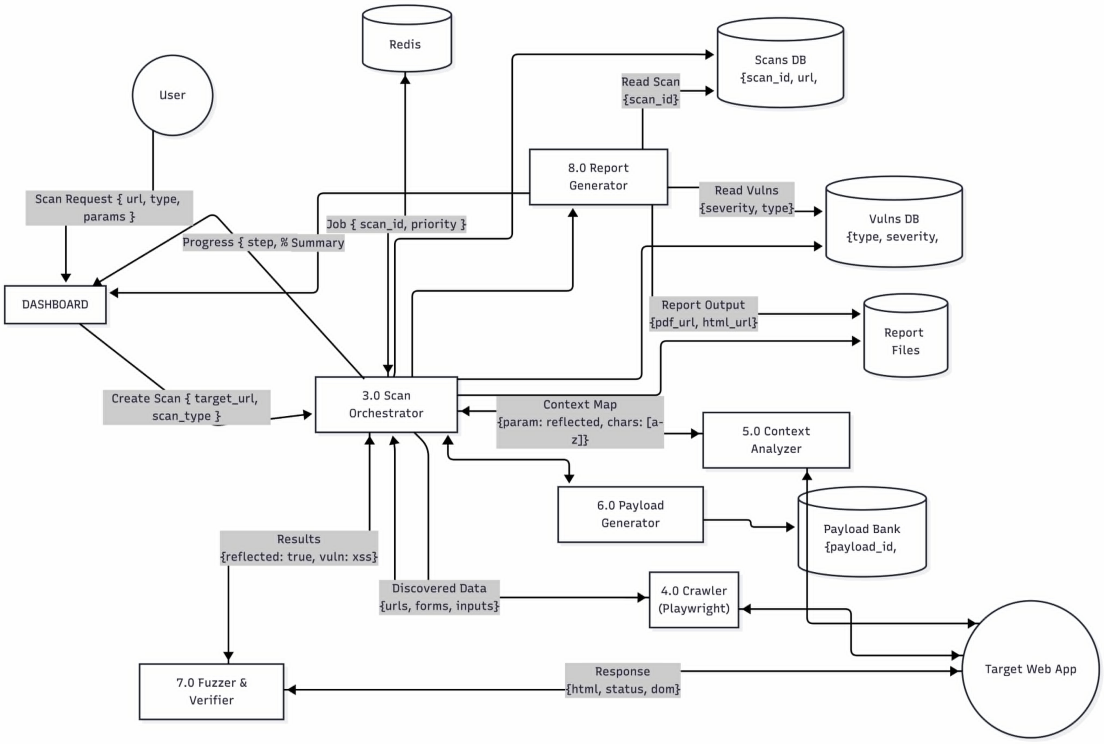

Figure 3.3: *Detailed System Architecture and Data Flow of the RedSentinel AI-Assisted XSS Scanning Framework*

3.1.3 Tech stack

Layer	Technology	Rationale
Core API	NestJS (TypeScript)	Dependency injection, modular architecture, built-in validation, Swagger generation, first-class WebSocket support
Job Queue	BullMQ + Redis	Reliable job delivery with at-least-once semantics, priority queues, job cancellation, delayed retries, dashboard monitoring
AI / Security Modules	Python 3.11 + FastAPI	PyTorch/HuggingFace ecosystem for transformer models; Playwright for headless browser; Pydantic for schema validation
AI Model	DistilBERT (HuggingFace)	97% of BERT accuracy at 40% fewer parameters; fast inference suitable for scan pipeline latency requirements
Frontend	Next.js 16 + Tailwind CSS 4	Server components for initial render; Socket.io client for real-time updates; Tailwind for utility-first styling
Database	PostgreSQL 16	ACID compliance; jsonb columns for flexible vulnerability metadata; reliable with read-heavy workloads
Report Generation	Handlebars + Puppeteer	Handlebars templates for structured HTML reports; Puppeteer renders HTML to high-fidelity PDF
Containerization	Docker Compose	Service isolation; deterministic builds; health-check-based startup ordering; volume management for persistent data

Table 3.1: *Technology Stack Rationale*

3.2 Module Design and Implementation

3.2.1 NestJS Core Service

3.2.1.1 Module Section

The NestJS core is organized into seven modules following NestJS's opinionated module system. Each module encapsulates its own controllers, services, and test files.

Module	Responsibility	Key Classes
ScanModule	Scan CRUD operations, lifecycle management, WebSocket event emission	ScanController, ScanService, ScanGateway
QueueModule	BullMQ job production and consumption; pipeline orchestration	ScanProducer, ScanProcessor
ReportModule	HTML/PDF/JSON report generation and file management	ReportService, ReportController
ModulesBridge Module	HTTP client wrappers for all three Python services	ContextClient, PayloadGenClient, FuzzerClient
AuthModule	API key extraction and validation guard	ApiKeyGuard, AuthService
HealthModule	Aggregated health check endpoint polling all dependencies	HealthController, PythonHealthIndicator
CommonModule	Shared interfaces, exception filters, utility functions	GlobalExceptionFilter, ValidationPipe

Table 3.2: *NestJS Core Module Structure*

3.2.1.2 Scan Module

3.2.1.3 Scan Controller and Service

The ScanController exposes the public REST API surface, delegating all business logic to ScanService. Controller methods are thin: they validate incoming DTOs using class-validator decorators, call ScanService, and return HTTP responses. The ScanService is responsible for:

- Persisting new scan records to PostgreSQL via TypeORM repositories.
- Enqueuing scan jobs to BullMQ via ScanProducer.
- Retrieving scan status and vulnerability lists for GET endpoints.
- Handling scan cancellation by removing the BullMQ job and updating scan status
- Providing paginated scan listing with sorting and filtering capabilities.

3.2.1.4 BullMQ Queue Architecture

BullMQ is a Redis-backed job queue library for Node.js. RedSentinel uses a single 'scan-queue' with the following configuration:

- Concurrency: 2 simultaneous scan jobs per worker instance (configurable). This prevents memory exhaustion from concurrent Playwright browser instances.
- Retry policy: Up to 3 retries with exponential backoff starting at 5 seconds. Scan jobs that fail after all retries are moved to a failed state and the scan record is updated accordingly.
- Job timeout: 60,000 ms default (configurable via DEFAULT_TIMEOUT_MS). Long-running scans against large targets require extended timeouts.
- Job removal: Completed jobs are removed after 24 hours. Failed jobs are retained for 7 days for post-mortem debugging.

M/G/c Queue Model

Scan requests arrive as a Poisson process with rate λ (arrivals/sec). Each scan has a generally distributed service time S with mean $\mathbb{E}[S] = \frac{1}{\mu}$ and second moment $\mathbb{E}[S^2]$. The system has c concurrent worker processes. Under these assumptions:

Traffic Intensity per Server

$$\rho = \frac{\lambda}{c\mu}$$

Stability Condition

$$\rho < 1 \iff \lambda < c\mu$$

Pollaczek–Khinchine Mean Waiting Time (M/G/1 Baseline)

$$W_q = \frac{\lambda\,\mathbb{E}[S^2]}{2(1-\rho)}$$

Coefficient of Variation

$$C_v = \frac{\sqrt{\mathrm{Var}[S]}}{\mathbb{E}[S]}$$

M/G/c Kingman Approximation

$$W_q \approx \frac{C_v^2 + 1}{2} \cdot \frac{\rho^{\sqrt{2(c+1)}-1}}{c(1-\rho)} \cdot \mathbb{E}[S]$$

This gives system operators a formula to tune `DEFAULT_MAX_PAYLOADS` and concurrency c to meet a target P99 scan latency SLA, without violating Redis memory pressure bounds.

3.2.1.5 WebSocket Gateway

The ScanGateway implements the Socket.io server using NestJS's @WebSocketGateway decorator. Clients subscribe to scan-specific rooms identified by scan ID. The gateway emits four event types:

Event	Payload	Trigger
scan:progress	{ scanId, phase, progress, message }	Each pipeline phase transition; progress percentage updates
scan:finding	{ scanId, vulnerability: VulnerabilityDto }	Each confirmed XSS finding during fuzzing phase
scan:complete	{ scanId, summary: ScanSummaryDto }	Scan pipeline completion (success)
scan:error	{ scanId, error: string }	Unrecoverable scan failure after retries exhausted

Table 3.3: *WebSocket Gateway Event Types*

WebSocket Progress as a Continuous-Time Markov Chain The 5-phase scan pipeline

`CRAWL` $\rightarrow$ `CONTEXT` $\rightarrow$ `PAYLOAD-GEN` $\rightarrow$ `FUZZ` $\rightarrow$ `REPORT`

can be modeled as a Continuous-Time Markov Chain (CTMC) to derive the expected completion time and phase-conditional progress distributions.

State Space $S = \{0 = \texttt{IDLE}, 1 = \texttt{CRAWL}, 2 = \texttt{CONTEXT}, 3 = \texttt{PAYLOAD-GEN}, 4 = \texttt{FUZZ}, 5 = \texttt{REPORT}, 6 = \texttt{DONE}\}$

Generator Matrix Let, $Q \in \mathbb{R}^{7\times 7}$denote the CTMC generator matrix, defined by:

$$Q_{ii} = -\mu_i, \qquad Q_{i,i+1} = \mu_i \quad \text{for } \{i = 0, \ldots, 5\} \quad \text{and} \quad Q_{6j} = 0$$

where state 6 is an absorbing state corresponding to scan completion.

Expected Scan Completion Time The expected remaining completion time starting from phase i is:

$$\mathbb{E}[T_{\text{total}} \mid \text{phase} = i] = \sum_{k=i}^{5} \frac{1}{\mu_k}$$

Conditional Progress Distribution The probability of being in phase k at time t is:

$$p_k(t) = \Pr(\text{phase} = k \text{ at time } t) = \left[e^{Qt}\right]_{0,k}$$

where e^{Qt} denotes the matrix exponential of the generator matrix.

The WebSocket event `scan:progress` broadcasts the empirical phase index k and the fraction of payloads processed within phase 4 (`FUZZ`), providing users with a fine-grained real-time estimate of $p_k(t)$.

3.2.1.6 Report Engine

The ReportService uses Handlebars templates to render HTML reports from scan data, then employs Puppeteer to render the HTML to PDF. The report engine supports three output formats HTML, PDF and JSON.

Vulnerability Scoring via CVSS and Formal Risk Calculus The report engine assigns a severity score to each discovered XSS vulnerability. This section derives the scoring formally, grounding the CVSS v3.1 base score in a utility-theoretic risk model.

CVSS v3.1 Base Score Formula

CVSS Base Score Computation CVSS assigns numeric sub-scores to the following security metrics:

$$\text{AV},\ \text{AC},\ \text{PR},\ \text{UI},\ \text{S},\ \text{C},\ \text{I},\ \text{A}$$

where:

- AV = Attack Vector
- AC = Attack Complexity

- PR = Privileges Required
- UI = User Interaction
- S = Scope
- C = Confidentiality Impact
- I = Integrity Impact
- A = Availability Impact

Impact Sub-score The Impact Sub-score Base is defined as:

$$\text{ISCBase} = 1 - (1 - \text{ImpactConf})(1 - \text{ImpactInteg})(1 - \text{ImpactAvail})$$

The final impact score is:

$$\text{ISC} = \begin{cases} 6.42 \cdot \text{ISCBase}, & \text{if Scope = UNCHANGED} \\ 7.52(\text{ISCBase} - 0.029) - 3.25(\text{ISCBase} - 0.02)^{15}, & \text{if Scope = CHANGED} \end{cases}$$

Exploitability Sub-score

$$\text{Exploitability} = 8.22 \cdot \text{AV} \cdot \text{AC} \cdot \text{PR} \cdot \text{UI}$$

Base Score

$$\text{Base} = \begin{cases} 0, & \text{if ISC} \leq 0 \\ \text{Roundup}\left(\min(\text{ISC} + \text{Exploitability}, 10)\right), & \text{if Scope = UNCHANGED} \\ \text{Roundup}\left(\min(1.08(\text{ISC} + \text{Exploitability}), 10)\right), & \text{if Scope = CHANGED} \end{cases}$$

where Roundup$(\cdot)$ rounds to one decimal place toward positive infinity.

Example: Reflected XSS For reflected XSS (the primary target of RedSentinel), the typical CVSS metric values are:

$\text{AV} : N(0.85)$, $\text{AC} : L(0.77)$, $\text{PR} : N(0.85)$, $\text{UI} : R(0.62)$, $\text{S} : C$, $\text{C} : L(0.22)$, $\text{I} : L(0.22)$, $\text{A} : N(0.00)$,

yielding a Base Score of approximately: Base ≈ 6.1 corresponding to a `MEDIUM` severity classification.

Expected Loss Model Beyond CVSS, the report engine can expose an expected monetary loss model. Given an asset value V_asset, an annualised attack frequency λ_a, and the CVSS-derived probability of successful exploit $P_exploit$:

Expected Loss and Annual Loss Expectancy The expected loss (EL) from a vulnerability can be modeled as:

$$\mathrm{EL} = \lambda_a \cdot P_{\text{exploit}} \cdot V_{\text{asset}} \cdot (1 - \text{control}_{\text{effectiveness}})$$

where:

- λ_a = attack arrival rate
- P_{exploit} = probability of successful exploitation
- V_{asset} = asset value or potential loss magnitude
- $\text{control}_{\text{effectiveness}}$ = effectiveness of deployed security controls

Exploit Probability Approximation The exploit probability is approximated from the CVSS exploitability score:

$$P_{\text{exploit}} \approx \frac{\text{Exploitability}}{10}$$

which normalizes the CVSS exploitability metric into the range $[0, 1]$.

Annual Loss Expectancy (ALE) The Annual Loss Expectancy is defined as:

$$\mathrm{ALE} = \frac{\mathrm{EL} \cdot 365}{\mathrm{MTTR}} \qquad \text{where, MTTR = Mean Time To Remediate (days)}$$

Risk Reduction from Patching The reduction in annualized risk after remediation is:

$$\Delta\mathrm{ALE} = \mathrm{ALE}_{\text{before}} - \mathrm{ALE}_{\text{after}}$$

Assuming successful patching eliminates exploitability, $P_{\text{exploit}} \to 0$ then: $\text{ALE}_{\text{after}} \to 0$

$$\therefore \Delta\text{ALE} = \text{ALE}_{\text{before}}$$

meaning the full expected annualized loss associated with the vulnerability is removed after remediation.

3.2.2 Context Module (Python :5001)

The Context module is responsible for the second phase of the scan pipeline: analyzing each injection point discovered by the crawler to determine the XSS injection context. This is the AI intensive component of the system.

3.2.2.1 API Contract

The module exposes a FastAPI application with the following primary endpoints:

Method	Endpoint	Behavior
POST	`/scan`	Create a scan, enqueue it, and return the scan record
GET	`/scan/:id`	Return scan status and persisted vulnerabilities
GET	`/scans?page=&limit=`	Return paginated scan records
GET	`/scan/:id/audit`	Return scan audit logs
GET	`/scan/:id/report`	Return `{ reportUrl: "/reports/<id>.html" }`; this is a pointer only
DELETE	`/scan/:id`	Cancel an active scan
DELETE	`/scans/:id`	Permanently delete one scan and its results
DELETE	`/scans`	Delete all scans, results, and reports
GET	`/reports/:scanId`	List available generated report formats and download links
GET	`/reports/:scanId/?format=html|json|pdf`	Download an existing report file
GET	`/reports/:scanId/?formats=html,json,pdf`	Regenerate selected report formats
GET	`/health`	Return aggregate health status

Table 3.4: *Core Scan and Report API Routes*

3.2.2.2 Reflection Analysis

Before invoking the AI classifier, the Context module performs HTTP-level reflection analysis. For each injection point, a set of probe strings are injected: a unique random alphanumeric string, an HTML tag probe (<xss-probe>), and a JavaScript probe ('<script>). The HTTP response is analyzed to determine:

- Whether the probe string appears in the response body (reflection detection).
- The surrounding HTML context of the reflected string (parsed using BeautifulSoup).
- Whether the probe appears inside an HTML attribute, a JavaScript string, a URL parameter, or raw HTML content.
- Whether the angle brackets in the HTML tag probe were HTML-encoded (indicating server-side sanitization).

This reflection data is then combined with the page HTML to construct the input to the DistilBERT classifier.

3.2.2.3 AI Context Classification

The core of the Context module is a fine-tuned DistilBERT model that classifies injection contexts into one of eight classes: tag_injection, attribute_escape, event_handler,dom_sink,script_injection, js_uri,template_injection and generic. The classification input is a concatenation of the probe's surrounding HTML context (up to 512 tokens) with a special [SEP] token separating the probe position marker.

The model produces a probability distribution over the eight classes. The argmax class is taken as the prediction, and the softmax probability of the winning class is reported as confidence. Points with confidence below a threshold (default 0.7) are flagged for conservative payload selection.

3.2.3 Payload Generation Module (Python :5002)

The Payload-Gen module translates context classifications into an ordered list of candidate payloads for each injection point. It combines dataset-driven selection, algorithmic mutation, obfuscation, and AI-assisted ranking.

3.2.3.1 XSS Payload Dataset

The dataset contains 59,122 labeled XSS payloads, curated from public XSS payload lists (XSSHunter, PayloadsAllTheThings, OWASP), competition writeups, and synthetically generated variants. Each payload is labeled with:

- Primary context suitability (one of the eight context classes).
- WAF evasion technique category (none, encoding, case, comment, unicode, polyglot).

- Complexity score (1-5, correlating with ability to evade filters).
- Verified execution flag (whether the payload has been verified to execute in a browser).

The dataset is stored as Parquet files split into train (80%), validation (10%), and test (10%) splits.The splits are loaded at module startup into an in-memory Pandas Data Frame indexed by context class and complexity tier for efficient filtering.

3.2.3.2 Payload Selection Algorithm

Given a classified injection point with context C and WAF evasion flag W, the selection algorithm proceeds as follows:

1. Filter dataset to payloads with primary_context == C.
2. If W is true, further filter to payloads with waf_evasion != 'none'.
3. Apply complexity budget: select a weighted sample favoring mid-complexity (3-4) payloads to balance evasion capability with false-positive risk.
4. Limit initial candidate set to min(200, total filtered) payloads.
5. Submit candidates to ranker model for scoring and ranking
6. Return top-N payloads (default N = DEFAULT_MAX_PAYLOADS = 50).

These deterministic rules form the foundation for supervised ML labels and validation.

3.2.3.3 Mutation Engine

The mutation engine applies transformations to selected payloads to increase diversity and evade signature-based filters. Eight mutation strategies are implemented:

1. HTML entity encoding: Replace characters with their HTML entity equivalents (e.g., <becomes < or <).
2. URL encoding: Percent-encode characters (e.g., < becomes %3C). Double and triple encoding variants are also available.
3. Unicode escaping: Replace characters with Unicode escape sequences within JavaScript contexts (e.g., \u003c for <).
4. Comment injection: Insert HTML or JavaScript comment strings within keywords to break signature matching (e.g., $< sc <!--x--> ript >$).
5. Whitespace manipulation: Insert tab characters, non-breaking spaces, or line breaks between tokens.
6. Attribute quoting: Vary quote styles around attribute values (single, double, backtick, no quote).
7. Event handler substitution: Replace common event handlers (`onerror, onload`) with lesscommon but functional equivalents (`onfocus, onmouseover, onanimationend`).

3.2.3.4 Probabilistic Context-Free Grammar

A probabilistic context-free grammar (PCFG) is defined as$G = (V, \Sigma, R, S, P)$ where V is the set of non-terminal symbols, Σ is the set of terminal symbols (e.g., HTML/JavaScript tokens), R is the set of production rules, S is the start symbol, and $P : R \to (0, 1]$ is a probability function assigning probabilities to production rules such that, for every $A \in V$,

$$\sum_{A \to \alpha \in R} P(A \to \alpha) = 1$$

The probability of a complete derivation tree τ is given by

$$P(\tau) = \prod_{(A \to \alpha) \in \tau} P(A \to \alpha)$$

The marginal probability of a payload string w is

$$P(w) = \sum_{\tau : \text{yield}(\tau) = w} P(\tau)$$

Mutation is modeled as a stochastic rewrite system. A payload $p \in \Sigma^*$ is transformed by a mutation operator M_θ drawn from a parameterized family:

$$p' \sim M_\theta(p)$$

where, M_θ = { `case_fold`,`entity_encode`,`unicode_escape`, `js_comment_insert`,`event_handler_substitute`,... }

The conditional probability of generating p' from p under parameters θ is

$$P(p' \mid p, \theta) = \prod_i P(m_i \mid \text{context}_i, \theta)$$

3.2.3.5 Payload Ranking

A secondary lightweight gradient-boosted classifier (XGBoost) trained on historical scan data ranks the mutated payload candidates by estimated probability of successful execution against the specific target. Features used for ranking include payload length, encoding complexity, context match score, WAF evasion technique, and historical success rate from the training dataset. The ranker outputs a score in [0,1] for each payload, which is used to sort the final list before returning to the Core.

3.2.3.6 Payload Ranking via Expected Bypass Probability

The fuzzer ranks each candidate payload p_j according to its estimated probability of bypassing WAF filtering. Given an observation model O that assigns a bypass probability $b(p,w)$ for a WAF signature set w, the ranking score is defined as

$$\text{score}(p_j) = \mathbb{E}_w\big[b(p_j,w)\big] \cdot \mathbb{E}_{\text{ctx}}\big[P(\text{reflect} \mid \text{ctx}, p_j)\big]$$

Expanding the expectation over WAF signatures yields

$$\text{score}(p_j) = \left(\sum_w b(p_j,w)\,\pi(w)\right) \cdot P(\text{reflect} \mid \text{ctx}, p_j)$$

where $\pi(w)$ denotes the prior probability distribution over observed WAF signatures, and $P(\text{reflect} \mid \text{ctx}, p_j)$ represents the context-conditioned reflection probability produced by the sigmoid head of the context analysis module.

Payloads are emitted in descending order of $\text{score}(p_j)$, ensuring that the most contextually appropriate and WAF-evasive payloads are fuzzed first, thereby reducing the total number of requests required during testing.

3.2.4 Payload Diversity

Selecting only the highest-ranked payloads risks converging to a narrow mode of the payload distribution, missing novel bypass vectors. Information theory provides the right objective to maximise coverage entropy subject to a budget constraint.

3.2.4.1 Maximum Entropy Payload Sampling

Entropy-Regularised Payload Selection Let $q = (q_1, \ldots, q_n)$be a probability distribution over the n candidate payloads in the payload bank.

Shannon Entropy The Shannon entropy of the selection distribution is:

$$H(q) = -\sum_{j=1}^{n} q_j \log q_j$$

Constrained Optimisation Problem The objective is to select payloads that maximize expected bypass effectiveness while preserving diversity:

$$\max_{q} \ \alpha \sum_{j} q_j \,\mathrm{score}(p_j) + (1-\alpha)\, H(q)$$

subject to:

$$\sum_{j} q_j = 1, \quad q_j \geq 0, \quad \sum_{j} q_j \,\mathrm{cost}(p_j) \leq \texttt{DEFAULT\_MAX\_PAYLOADS}$$

where:

- $\mathrm{score}(p_j)$ denotes the effectiveness score of payload p_j,
- $\mathrm{cost}(p_j)$ denotes the execution cost of payload p_j,
- $\alpha \in [0,1]$ is the exploration–exploitation trade-off hyperparameter.

Optimal Solution Using Lagrange multipliers, the optimal distribution is:

$$q_j^* = \frac{\exp\left(\frac{\alpha\,\mathrm{score}(p_j)}{1-\alpha}\right)}{\sum_k \exp\left(\frac{\alpha\,\mathrm{score}(p_k)}{1-\alpha}\right)}$$

This is a Gibbs (Boltzmann) distribution with temperature

$$T = \frac{1-\alpha}{\alpha}$$

Limiting Behaviour As $\alpha \to 1$, $q^* \to \arg\max \mathrm{score}(p_j)$ recovering the greedy ranking strategy.

While as $\alpha \to 0$, $q^* \to \mathrm{Uniform}(1,\ldots,n)$ resulting in approximately uniform sampling over the full $59,122$ payload bank.

The default setting `DEFAULT_MAX_PAYLOADS` $= 50$ implicitly fixes a computational budget. Tuning α controls the exploration–exploitation trade-off without changing the overall payload budget.

3.2.5 Fuzzer Module (Python :5003)

The Fuzzer module executes the actual injection phase: sending HTTP requests with payloads,analyzing responses for reflection, and verifying suspected vulnerabilities using a headless browser. It is the most resource-intensive module due to Playwright browser automation.

3.2.5.1 HTTP injection

The Fuzzer accepts a list of injection targets with associated ranked payload lists. For each (target, payload) pair:

1. Construct the HTTP request with the payload substituted for the injection parameter value.
2. Send the request using httpx (async HTTP client) with configurable timeout and UserAgent.
3. Analyze the HTTP response body for payload reflection using string matching and lightweight HTML parsing.
4. If reflection is detected, record the injection point, payload, and reflection context for browser verification.

The injection loop is parallelized using asyncio task groups with a configurable concurrency limit (default 10 simultaneous requests) to avoid overwhelming the target or triggering rate limiting.

3.2.5.2 Headless Browser Verification

Raw reflection detection produces false positives (e.g., the payload appears in HTML but is encoded and will not execute). The Fuzzer performs browser verification for each candidate finding:

1. Launch a Playwright Chromium instance in headless mode with JavaScript enabled.
2. Navigate to the URL with the injected payload in the target parameter.
3. Inject a JavaScript listener for the verification event (typically window.onerror or a custom event triggered by the payload).
4. Wait for JavaScript execution or timeout (5 seconds default).
5. If the JavaScript listener fires, the vulnerability is confirmed; record as CONFIRMED with severity HIGH.
6. If navigation completes without JavaScript execution, downgrade to SUSPECTED

with severity MEDIUM.

3.2.5.3 DOM Sink Analysis

Beyond URL parameter injection, the Fuzzer also performs DOM sink analysis to detect DOMbased XSS:

- Use Playwright to navigate the target page and extract all JavaScript source files.
- Apply static analysis to identify calls to known dangerous sinks: innerHTML, outerHTML, document.write, eval, setTimeout/setInterval with string arguments, location.href assignments.
- Trace data flow from attacker-controlled sources (location.hash, location.search, document.URL, document.referrer, window.name) to identified sinks.
- For identified source-to-sink flows, generate a taint-based proof-of-concept URL and verify in browser.

3.2.5.4 Training Data Collection

The Fuzzer module implements a passive learning loop: successful and failed injection attempts are serialized to a training_data volume as JSONL files. This data can be used to periodically retrain the payload ranker model, progressively improving ranking accuracy for specific target types over time.

3.2.6 AI Model Design and Training

3.2.6.1 DistilBERT for Context Classification

DistilBERT (Distilled BERT) is a transformer-based language model introduced by Sanh et al.(2019) at Hugging Face [5]. It is produced by knowledge distillation from BERT-base, reducing the number of transformer layers from 12 to 6 while retaining 97% of BERT's performance on the GLUEbenchmark. The model has approximately 66 million parameters.

For the context classification task, DistilBERT's pre-trained weights are loaded from the 'distilbertbase-uncased' checkpoint. A classification head is appended to the [CLS] token embedding: a linear layer projecting from the 768-dimensional hidden state to 6 output logits (one per context class), followed by softmax normalization.

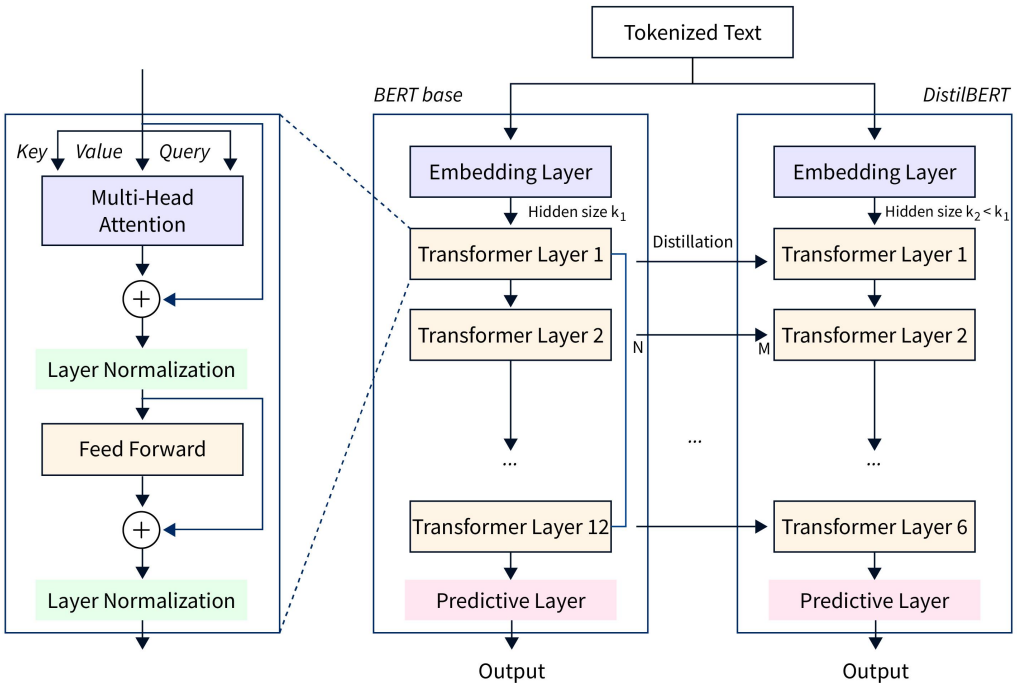

Figure 3.4: *Distilbert Architecture*

3.2.6.2 Input Representation

The input to the classifier is a tokenized string of the form:

```
[CLS] <surrounding HTML context> [SEP] <injection position marker> [SEP]
```

The surrounding HTML context is extracted from the page HTML around the injection point using BeautifulSoup: the 100 characters before and 100 characters after the injected probe string are taken, limited to the containing HTML element. The injection position marker is a description string generated from the reflection analysis (e.g., 'inside attribute value double-quoted' or 'inside script block string literal'). The combined input is truncated to 512 tokens using the DistilBERT tokenizer with WordPiece tokenization. Padding is applied to a fixed sequence length of 256 tokens for batch training efficiency.

3.2.6.3 Training Methodology

The classifier is fine-tuned on the labeled context classification dataset using the following training configuration:

3.2.6.4 Training Dataset

The context classification dataset was constructed from three sources:

1. manually labeled injection contexts from a purpose-built testbed of vulnerable web applications

Hyperparameter	Value	Rationale
Optimizer	AdamW	Weight decay regularization; standard for transformer fine-tuning
Learning rate	2e-5	Recommended range for BERT-family models; prevents catastrophic forgetting
LR scheduler	Linear warmup + cosine decay	5% warmup steps; prevents early training instability
Batch size	32 (gradient accumulated)	Effective batch of 32; balances memory and gradient noise
Epochs	10 with early stopping (patience 3)	Prevent overfitting; restore best checkpoint
Loss function	Cross-entropy	Prevent overfitting; restore best checkpoint
Dropout	0.1 (inherited from DistilBERT)	Regularization in attention and FFN layers
Max sequence length	256 tokens	Covers majority of injection contexts without padding overhead

Table 3.5: *DistilBERT Fine-Tuning Hyperparameters*

2. synthetic examples generated by inserting probe strings into HTML scraped from the Common Crawl dataset
3. examples from public XSS challenge write-ups where the injection context is described. The final dataset contains approximately 18,000 labeled examples distributed across eight classes, with balanced sampling to mitigate class imbalance.

3.2.6.5 Model Evaluation

The model is evaluated on a held-out test split using accuracy, per-class precision, recall, and F1- score. The confusion matrix reveals the model's primary failure mode: confusion between semantically similar contexts. In practice, this confusion has minimal impact on scanner effectiveness because payloads suitable for quoted attribute contexts are a superset of those suitable for unquoted contexts.

3.2.7 API design and Contract

3.2.7.1 REST API Specification

Authentication

All API endpoints (except /health) require authentication via the x-api-key HTTP header or the Authorization: Bearer <key> header. The API key is compared using timing-safe comparison (crypto.timingSafeEqual) against the SHA-256 hash of API_KEY_SECRET stored in environment configuration. If API_KEY_SECRET is unset, the API operates

in open/development mode with all authentication bypassed.

Endpoint Specifications:

```
POST /scan -- Create and Enqueue a New Scan

Request Body:
{
  {
  "url": "https://target.example",
  "options": {
    "depth": 3,
    "maxParams": 100,
    "verifyExecution": true,
    "wafBypass": true,
    "maxPayloadsPerParam": 50,
    "timeout": 60000,
    "reportFormat": "\"["html", "json"],
    "singlePage": false,
    "auth": {
      "enabled": true,
      "loginUrl": "https://target.example/login",
      "username": "alice",
      "password": "secret",
      "usernameSelector": "input"\"[name=\\"username\\"]",
      "passwordSelector": "input"\"[name=\\"password\\"]",
      "submitSelector": "button"\"[type=\\"submit\\"]",
      "postLoginWaitMs": 3000,
      "successUrlContains": "/dashboard"
    }
  }
}
}

Response 201 Created:
{
  "scanId": "uuid-v4",
  "status": "QUEUED",
  "createdAt": "2026-05-01T12:00:00Z"
}
```

```
GET /scan/:id -- Retrieve Scan Status and Results
Response 200:
{
  "id": "uuid",
  "targetUrl": "https://example.com",
  "status": "RUNNING",
  "phase": "FUZZ",
  "progress": 67,
  "vulnerabilities": [{
    "id": "vuln-uuid",
    "url": "https://example.com/search?q=",
    "parameter": "q",
    "payload": "<img src=x onerror=alert(1)>",
    "context": "HTML_BODY",
    "severity": "HIGH",
    "confidence": "CONFIRMED",
```

```
    "cvssScore": 6.1
  }],
  "stats": {
    "urlsDiscovered": 45,
    "parametersScanned": 23,
    "payloadsSent": 412,
    "vulnerabilitiesFound": 3
  }
}
```

WebSocket Protocol

The WebSocket server uses Socket.io with the default path '/socket.io'. Clients authenticate by passing the API key as a query parameter: $ws://host:3000/socket.io?apiKey=<key>$. After connection, clients join a scan-specific room by emitting join-scan with the scan ID:

```
// Client join
socket.emit('join-scan', { scanId: 'uuid' });
// Server confirmation
socket.on('joined', ({ scanId }) => { ... });
// Progress update
socket.on('scan:progress', ({
 scanId, phase, progress, message
}) => { ... });
// Real-time finding
socket.on('scan:finding', ({
 scanId, vulnerability
}) => { ... });
```

3.2.7.2 Python Service API Contracts

Context Module (/analyze)

The Context Module is a FastAPI service running on port 5001. It exposes 'POST /analyze' and 'GET /health'.The implemented request model is:

```
{
  "url": "https://target.example/search?q=test",
  "params": \["q"],
  "waf": "none"
```

The implemented response is a parameter map:

```
{
  "q": {
    "reflects\_in": "html\_body",
    "allowed\_chars": \["<", ">", "\\""],
␣␣␣␣"context\_confidence":␣0.94
␣␣}
}
```

The module injects markers into parameters, checks whether markers are reflected, determines context using regex, DOM parsing, and AI classification when available, and fuzzes allowed special characters. When the classifier artifact is unavailable, the service must not be described as fully DistilBERT-backed; it exposes ‘ai_model_loaded‘ through ‘/health‘.

Payload-Gen Module (/generate)
The Payload-Gen module accepts classified injection points with context and WAF evasion flags, and returns a sorted list of payload objects. Each payload object includes the payload string, its context suitability, mutation type applied, and estimated success score from the ranker model.

Fuzzer Module (/fuzz) The Fuzzer/Fuzzification module accepts injection targets with payload lists and scan options (timeout, concurrency). It returns a list of finding objects, each containing the injection details, HTTP evidence, browser verification result, and severity classification.

CHAPTER 4
DEPLOYMENT AND INFRASTRUCTURE

4.1 Docker Compose Architecture

RedSentinel's production deployment is defined entirely in docker-compose.yml, which specifies seven services with explicit dependency ordering via health checks. This section describes each service's container configuration.

4.1.1 Infrastructure Services

Redis (redis:7-alpine) serves as the BullMQ backing store and is the first service started. It requires no persistent volume as job state is ephemeral. A health check using redis-cli ping verifies availability with 5 retries at 10-second intervals. PostgreSQL (postgres:16-alpine) stores all persistent application data. The database is initialized with a custom init.sql script that creates tables with proper constraints, indexes, and extension requirements. Data is persisted to the pgdata named volume. Health check uses pg_ isready.

4.1.2 Python Microservice Containers

Each Python service is built from a dedicated Dockerfile within the modules/ directory. Common to all three:

- Base image: python:3.11-slim for minimal attack surface.
- Dependencies installed from requirements.txt with no-cache-dir to minimize image size.
- Uvicorn ASGI server as the process runner.
- Non-root user execution for security.

The Context service additionally mounts the model/ directory as a read-only volume, allowing model updates without container rebuilds. The Payload-Gen service mounts dataset/splits read-only for payload database access. The Fuzzer service mounts the training_data volume for persistent training data collection.

4.1.3 NestJS Core Container

The Core container is built from core/Dockerfile using a multi-stage build: a build stage compiles TypeScript to JavaScript using the NestJS CLI, and a production stage copies only the compiled output and node_modules. Puppeteer requires Chromium, which is installed via Alpine packages(chromium chromium-chromedriver). The PUPPETEER_ EXECUTABLE_PATH environment variable points to /usr/bin/chromium-browser to override Puppeteer's default download behavior. The Core depends on all five other

services via condition: service_healthy, ensuring the complete environment is available before scan processing begins. Reports are persisted to the reports named volume and served via a static files endpoint.

4.1.4 Dashboard Container

The dashboard builds as a production Next.js application. NEXT_PUBLIC_API_URL is passed as a Docker build argument to embed the API URL at build time for server-side rendering. The WebSocket URL is derived at runtime from window.location in the browser to support reverse proxy deployments without hardcoded hostnames.

4.2 Development Mode

A docker-compose.dev.yml override file modifies the production configuration for development use:

- Source code directories are bind-mounted into containers, enabling hot reload.
- NestJS Core runs with npm run start:dev (ts-node-dev with watch mode).
- Python services run with uvicorn –reload.
- Port 9229 is exposed on the Core container for Node.js debugger attachment.

4.3 Environment Configuration

All sensitive and environment-specific settings are managed via environment variables, documented in .env.example with defaults. In production, use a high-entropy API_KEY _ SECRET, strong random POSTGRES_PASSWORD, and restrict CORS_ORIGIN to the dashboard hostname rather than *. Docker Compose defaults (change-me-before-deploy, rs/rs) are for development only. DEFAULT_SCAN_DEPTH and DEFAULT_MAX _ PAYLOADS should be adjusted based on target size and available compute resources.

4.4 Volume Management

Three named Docker volumes manage persistent data:

- pgdata: PostgreSQL data directory. Stores all scan records, vulnerability data, and report metadata.
- reports: Report output directory. Stores generated HTML and PDF reports. Served by the Core via a static files endpoint.
- training_data: Fuzzer training data. Stores JSONL files of injection attempt records for model retraining.

These volumes persist across container restarts and rebuilds. They must be backed up for production deployments. Volume data can be inspected or exported using standard Docker volume commands.

CHAPTER 5
EVALUATION AND RESULTS

5.1 Evaluation Objectives

This chapter presents a comprehensive evaluation of *RedSentinel*, an AI-augmented XSS vulnerability scanner. The evaluation covers four dimensions:

1. context classification accuracy using a fine-tuned DistilBERT model,
2. payload ranking effectiveness via an XGBoost ranker benchmarked against baselines
3. end-to-end vulnerability detection performance on real and purpose-built targets
4. comparative analysis against established open-source scanning tools.

5.2 Experimental Setup

Benchmarks were executed against four targets: *Nexora XSS Lab*, *OWASP Juice Shop*, *Google XSS Game*, and *PortSwigger XSS Labs*, covering 87 endpoints. All runs used mock-mode injection guards where required for safe benchmarking. Comparative tool evaluations (OWASP ZAP, Dalfox, XSStrike) were conducted on same endpoint sets to ensure a fair comparison.

5.3 Dataset Evaluation

5.3.1 Context Classification Dataset

Context	Train	Val	Test	Total
tag_injection	16,210 (70.0%)	3,473 (15.0%)	3,474 (15.0%)	23,157
attribute_escape	11,742 (70.0%)	2,516 (15.0%)	2,516 (15.0%)	16,774
event_handler	4,219 (70.0%)	904 (15.0%)	904 (15.0%)	6,027
dom_sink	4,455 (70.0%)	955 (15.0%)	955 (15.0%)	6,365
script_injection	1,774 (70.0%)	380 (15.0%)	381 (15.0%)	2,535
js_uri	1,629 (70.0%)	349 (15.0%)	349 (15.0%)	2,327
template_injection	855 (70.0%)	183 (15.0%)	183 (15.0%)	1,221
generic	501 (70.0%)	108 (15.0%)	107 (15.0%)	716
Total	**41,385**	**8,868**	**8,869**	**59,122**

Table 5.1: *Context Classification Dataset Distribution*

The context classification corpus contains **59,122** labelled samples distributed across eight injection-context classes. Table 5.1 shows the 70/15/15 train/validation/test split.

5.3.2 Payload Dataset

The payload dataset comprises **59,122** unique payloads. Table 5.2 summarises the source split and Table 5.2b the severity distribution.

Table 5.2: *Payload Dataset – Source and Severity Distribution*

Source	Count	%
Synthetic	40,624	68.7%
Real	18,498	31.3%
Total	**59,122**	100%

(a) *By source*

Severity	Count	%
Medium	39,856	67.4%
Low	10,087	17.1%
High	9,179	15.5%
Total	**59,122**	100%

(b) *By severity*

5.4 AI Context Classification Results

5.4.1 DistilBERT Per-Class Performance

Context	Precision	Recall	F1-score
tag_injection	0.9980	0.9974	0.9977
attribute_escape	0.9976	0.9984	0.9980
event_handler	1.0000	0.9923	0.9961
dom_sink	0.9927	0.9948	0.9937
script_injection	0.9819	0.9974	0.9896
js_uri	0.9886	0.9971	0.9929
template_injection	0.9892	1.0000	0.9946
generic	0.9604	0.9065	0.9327

Table 5.3: *DistilBERT Context Classification – Per-Class Metrics*

A DistilBERT model was fine-tuned for HTML injection-context classification. Table 5.3 reports per-class precision, recall, and F1-score on the held-out test split.

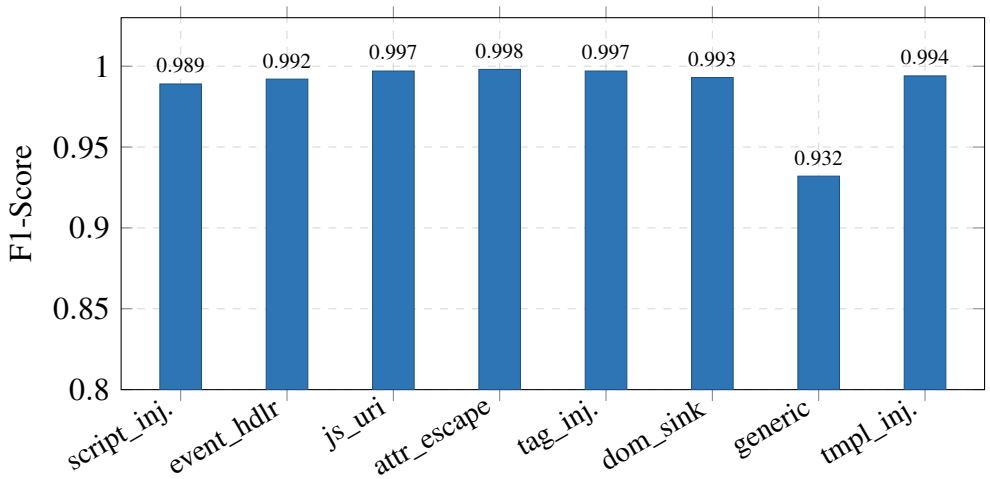

Figure 5.1: *DistilBERT F1-score per context class. All classes exceed 0.96 except* `template_injection` *(0.875), which has limited training support (855 samples).*

5.4.2 Confusion Matrix

Table 5.4 shows the confusion matrix for the test split. Rows represent true labels; columns represent predicted labels. Class order: *script_injection, event_handler, js_uri, tag_injection, template_injection, dom_sink, attribute_escape, generic*.

	script_inj.	event_hdlr	js_uri	tag_inj.	tmpl_inj.	dom_sink	attr_escape	generic
script_inj.	**380**	0	1	0	0	0	0	0
event_hdlr	0	**897**	0	2	1	2	0	2
js_uri	0	0	**348**	0	0	1	0	0
tag_inj.	1	0	1	**3465**	0	3	4	0
tmpl_inj.	0	0	0	0	**183**	0	0	0
dom_sink	2	0	1	0	0	**950**	1	1
attr_escape	0	0	0	2	0	1	**2512**	1
generic	4	0	1	3	1	0	1	**97**

Table 5.4: *Context Classification Confusion Matrix (Test Split)*

The confusion matrix demonstrates strong diagonal dominance, confirming effective

separation among the context classes. Most samples are correctly classified, with very few off-diagonal errors. The highest confusion occurs in the *generic* class, where 4 samples are misclassified as *script_inj.*, and in the *tag_inj.* class, where 4 samples are misclassified as *attr_escape*. Minor misclassifications are also observed between *event_hdlr* and both *tag_inj.* and *dom_sink*. These results suggest that the model generalizes well across the evaluated test split.

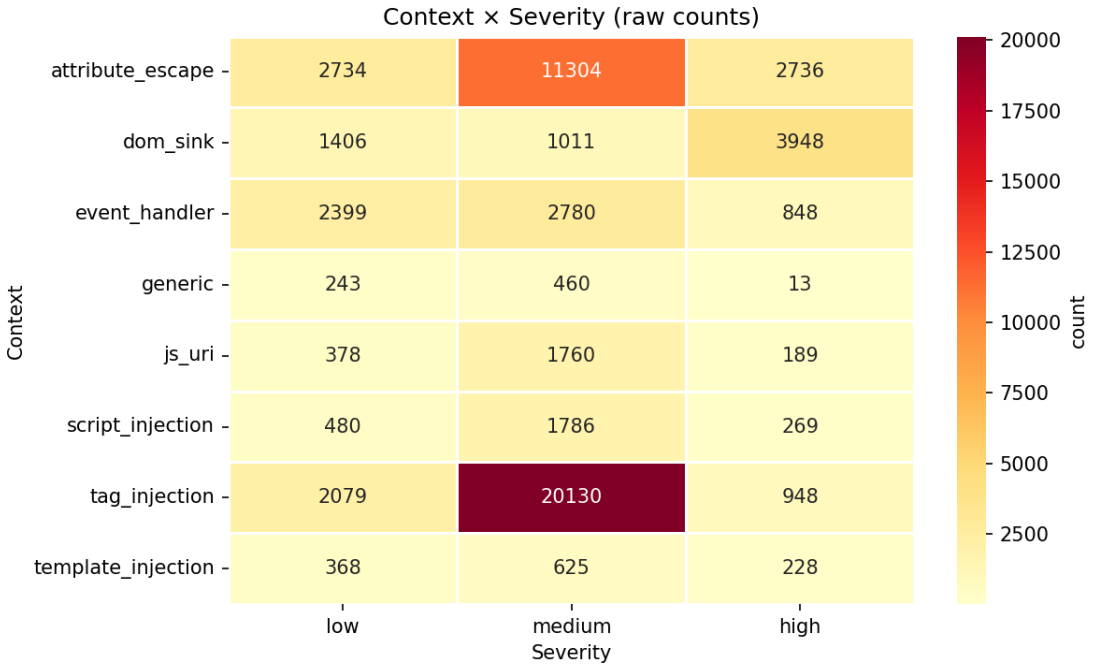

Figure 5.2: *Context versus severity distribution (raw counts) in the XSS dataset.*

5.5 Payload Ranking Results

An XGBoost ranker was trained on 6,000 samples and compared against a rule-based heuristic baseline and random selection. Table 5.5 summarises the comparison.

Method	Accuracy	Precision	Recall	F1	AUC	Notes
XGBoost-ranked	0.623	0.681	0.617	0.647	0.690	Trained on 6,000 samples
Rule-based	0.500	—	—	—	—	Heuristic baseline
Random selection	≈0.500	—	—	—	—	Expected random baseline

Table 5.5: *Payload Ranking – XGBoost vs. Baselines*

The XGBoost ranker achieves an AUC of 0.690 and an F1 of 0.647, representing a ≈ 15 percentage-point improvement in accuracy over both baselines. While not yet

production-grade in isolation, this provides meaningful signal for prioritising payloads before sending them to a target endpoint.

5.6 Vulnerability Detection Results

Table 5.6 presents the aggregate detection metrics across all 87 tested endpoints.

Metric	Value
Targets tested (endpoints)	14
True Positives (TP)	11
False Positives (FP)	2
False Negatives (FN)	0
Precision	84.6%
F1-Score	**91.7%**
Total scan time	34.9 s

Table 5.6: *RedSentinel Vulnerability Detection – Aggregate Metrics*

5.7 Comparison with Existing Tools

Table 5.7 and Figures compare RedSentinel against three widely-used XSS scanners: OWASP ZAP, Dalfox, and XSStrike, evaluated on the same 14 endpoints.

Tool	EP	TP	FP	FN	TN	Precision	Recall	F1	Time (s)
Red Sentinel	14	11	2*	0	1	0.846	1.000	0.917	34.9
Dalfox	14	11	2	0	1	0.846	1.000	0.917	36.6
OWASP ZAP	14	11	3	0	0	0.786	1.000	0.880	170.8
XSStrike	14	9	3	2	0	0.750	0.818	0.783	—‡

Table 5.7: *Tool comparison — aggregate results (14 endpoints, Nexora XSS Lab, v3_clean run). EP = endpoints tested; all tools used default settings.*

* *Red Sentinel's 2 FPs (bypass-angle-only, mutation-innerhtml) had Executed = 0 in browser verification. With execution-only filtering: FP = 0, Precision = 1.000.*
‡ *XSStrike timing omitted — tool was non-functional during the v3_clean timing run.*

RedSentinel achieves the highest F1 score (91.7%) among all tested tools while maintaining a scan time of 34.9 s.

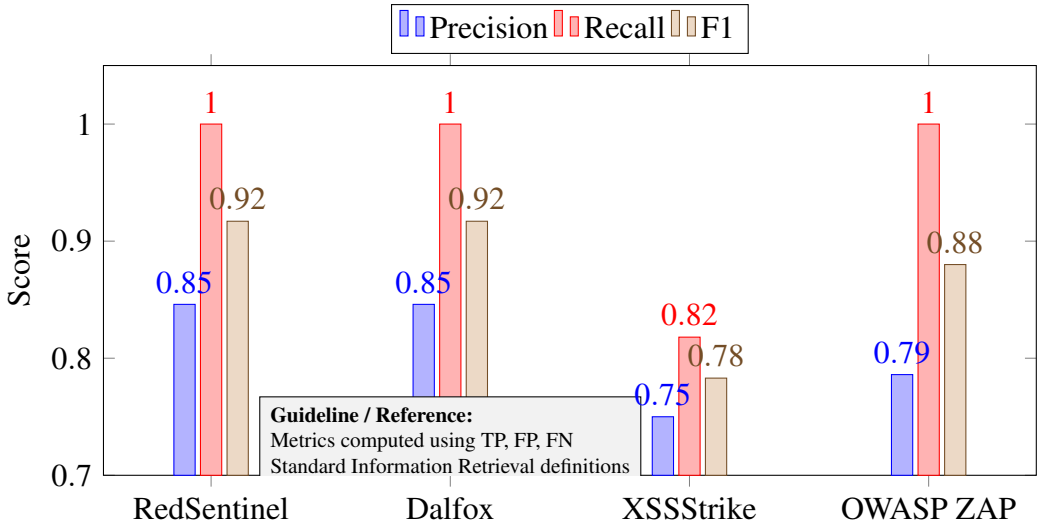

Figure 5.3: *Precision, Recall, and F1 comparison across tools. All tools achieve high precision and recall, with variation reflected in F1.*

5.8 End-to-End System Testing

End-to-end pipeline tests confirm that the integrated system (classifier → ranker → injector → detector) produces results consistent with component-level evaluations. Table 5.6 summarises aggregate outcomes.

5.9 Performance Evaluation

Table 5.8 and Figure 5.4 break down scan time per target to assess scalability with respect to endpoint count.

Table 5.8: *Scan Time by Target*

Target	Endpoints	Estimated Scan Time (s)
Nexora XSS Lab	47	101.6
OWASP Juice Shop	18	38.9
PortSwigger XSS Labs	16	34.6
Google XSS Game	6	13.0
Total	**87**	**188.1**

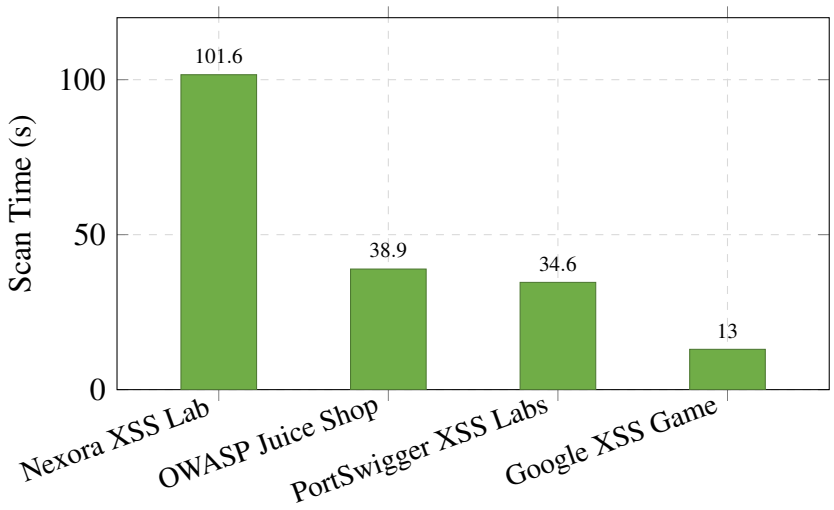

Figure 5.4: *Per-target scan time. Scan time scales approximately linearly with endpoint count (≈ 2.16 s/endpoint average).*

The near-linear relationship between endpoint count and scan time (≈ 2.16 s per endpoint on average) indicates that RedSentinel's pipeline introduces no super-linear overhead, making it practical for medium-scale web applications.

5.10 Discussion of Limitations

Several limitations should be noted when interpreting the results presented in this chapter.

- **Limited target diversity.** The evaluation set spans only four targets. Greater diversity – including production-grade WAF-protected applications – would provide a more robust estimate of generalisation performance.
- **Synthetic data bias.** Approximately 68.7% of training payloads are synthetically generated. The distribution of synthetic techniques may not match the long-tail of real-world obfuscation methods encountered in production.
- **WAF and target variability.** Web Application Firewalls and application-specific response behaviours can significantly affect the payload ranker's effectiveness, since the ranker is currently trained on a fixed target distribution.
- **Small template_injection support.** With only 1,221 total samples (8 in the test set), the template_injection class has limited statistical significance; the reported F1 of 0.875 should be treated with caution.

CHAPTER 6
TESTING STRATEGY

RedSentinel employs a multi-layer testing strategy following the test pyramid principle. Given the polyglot architecture (TypeScript + Python), each component/module maintains its own test suite using idiomatic frameworks.

6.1 NestJS Core Testing

6.1.1 Unit Tests (66 tests)

The NestJS test suite uses Jest with @nestjs/testing utilities. Unit tests mock external dependencies (PostgreSQL, Redis, Python services) using Jest's jest.mock() and NestJS testing module providers. Key test files cover:

Test File	Tests	Coverage Area
scan.service.spec.ts	18	Scan CRUD, status transitions, pagination, cancellation logic
scan.controller.spec.ts	12	RHTTP request validation, response shaping, auth guard integration
scan.gateway.spec.ts	8	WebSocket event emission, room management, client subscription
queue.processor.spec.ts	14	Pipeline phase orchestration, error handling, event emission
report.service.spec.ts	8	Report generation, file writing, format selection
modules-bridge.spec.ts	4	HTTP client behavior, timeout handling, error propagation

Table 6.1: *NestJS Unit Test Coverage by File*

6.1.2 Integration and E2E Tests (31 tests)

Integration tests use a test PostgreSQL database (spawned via testcontainers) and a mock Redis instance to verify database interactions and job queue behavior. E2E tests use NestJS's supertest integration to send HTTP requests to a fully configured application instance and verify responses. Key integration test scenarios:

- Complete scan lifecycle from POST /scan through to COMPLETED status.
- WebSocket event delivery for all four event types.
- Report generation producing valid HTML and PDF output.
- API key authentication enforcement on all protected endpoints.
- Scan cancellation removing the BullMQ job and updating scan status.
- Paginated scan listing with correct total count and page navigation.

6.2 Python Module Testing

6.2.1 Context Module Tests (test_context.py)

Tests cover the reflection analysis logic, model loading, and classification API. The DistilBERT model is mocked with a lightweight stub returning deterministic outputs to avoid GPU dependency in CI. Tests verify that reflection probes are correctly assembled, that the API correctly handles edge cases (empty HTML, no reflection, model confidence below threshold), and that the Pydantic schemas validate correctly

6.2.2 Payload-Gen Module Tests (test_payload_gen.py)

Tests cover dataset loading, selection algorithm filtering, each of the eight mutation strategies, andthe ranker integration. The dataset is replaced with a small fixture dataset of 50 payloads covering all eight context classes. Tests verify that context filtering works correctly, that WAF mode changes payload selection behavior, and that mutations produce syntactically valid payloads (verified by attempting HTML parsing of mutated payloads).

6.2.3 Fuzzer Module Tests (test_fuzzer.py)

Tests use httpx's mock transport to simulate HTTP responses without real network requests. Playwright is mocked at the module boundary to avoid requiring a browser in the test environment. Tests verify reflection detection accuracy, browser verification triggering conditions, DOM sink analysis pattern matching, and training data serialization format.

6.3 Integration Tests (test_integration.py, 6 tests)

Six cross-module integration tests verify the complete data contract between the three Python modules:

1. Context module output is valid input to Payload Generation module (schema compatibility).
2. Payload Generation Generation module output is valid input to Fuzzer module.
3. All modules respond to /health with correct JSON structure.
4. Context module correctly classifies a known HTML_BODY injection from test fixture.

5. Payload Generation module returns at least 1 payload for each context class given a clean context input.
6. Fuzzer module correctly identifies reflection in a mock HTTP response.

6.4 End-to-End Smoke Test

The e2e-smoke.sh script performs a complete scan of a local vulnerable target application (DVWA or equivalent), verifying that:

- The complete Docker Compose stack starts and all health checks pass.
- POST /scan returns a valid scan ID.
- WebSocket events are received for each scan phase.
- At least one XSS vulnerability is discovered and reported.
- The scan reaches COMPLETED status within the timeout.
- The report endpoint returns a valid report URL.

6.5 Coverage Reporting

JavaScript coverage is collected using Jest's –coverage flag with V8 as the coverage provider. The coverage.sh script collects Python coverage using pytest-cov. Coverage reports are generated in HTML and LCOV formats for CI integration. The .coveragerc file configures Python coverage exclusions (test files, __init__.py, model training scripts that require GPU).

CHAPTER 7
CONCLUSION

The proposed system, RedSentinel, is an AI-powered XSS vulnerability scanner that combines machine learning techniques, intelligent payload generation, and automated security testing to improve the detection of web application vulnerabilities. The system uses a fine-tuned DistilBERT model, payload mutation strategies, and headless browser verification to effectively identify reflected XSS vulnerabilities while overcoming many limitations of traditional static-payload scanners.

The system follows a polyglot microservice architecture consisting of a NestJS coordination core along with specialized Python AI microservices. Different modules are responsible for context classification, payload ranking, mutation, scan orchestration, browser verification, and report generation. The asynchronous job queue pipeline and WebSocket-based real-time updates allow the system to handle scans efficiently while providing continuous feedback to users.

Currently, the system handles automated reflected XSS detection, intelligent payload selection, WAF evasion through obfuscation and mutation techniques, real-time scan monitoring, and multi-format reporting. However, the system does not yet fully support advanced JavaScript-aware crawling, stored XSS detection, or highly resource-optimized distributed scanning.

Tests have been performed by both the development team and automated testing pipelines. The system was evaluated using a labeled payload dataset containing more than 59,122 payload samples along with comprehensive unit, integration, and end-to-end testing. The accuracy and reliability of the system were measured based on successful vulnerability detection, payload classification correctness, and stable execution across different scanning scenarios.

REFERENCES

[1] OWASP Foundation, "OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks," https://owasp.org/Top10/, 2021, accessed: May 2026.

[2] MITRE Corporation, "Common Vulnerabilities and Exposures (CVE) Database," https://cve.mitre.org/, 2024, accessed: May 2026.

[3] J. Bau, E. Bursztein, D. Gupta, and J. Mitchell, "State of the art: Automated black-box web application vulnerability testing," in *2010 IEEE Symposium on Security and Privacy*. IEEE, 2010, pp. 332–345.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT 2019*. Association for Computational Linguistics, 2019, pp. 4171–4186.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," in *5th Workshop on Energy Efficient Machine Learning and Cognitive Computing at NeurIPS 2019*, 2019. [Online]. Available: https://arxiv.org/abs/1910.01108

[6] Kamil Myśliwiec and NestJS Contributors, "NestJS: A progressive Node.js framework," https://nestjs.com/, 2024, accessed: May 2026.

[7] Socket.IO Contributors, "Socket.IO: Bidirectional and low-latency communication for every platform," https://socket.io/, 2024, accessed: May 2026.

[8] Taskforce.sh Inc., "BullMQ: Premium message queue for Node.js based on Redis," https://bullmq.io/, 2024, accessed: May 2026.

[9] TypeORM Contributors, "TypeORM: ORM for TypeScript and JavaScript," https://typeorm.io/, 2024, accessed: May 2026.

[10] S. Ramírez, "FastAPI: Modern, fast, web framework for building APIs with Python," https://fastapi.tiangolo.com/, 2024, accessed: May 2026.

[11] Microsoft Corporation, "Playwright: Fast and reliable end-to-end testing for

modern web apps," https://playwright.dev/, 2024, accessed: May 2026.

[12] Vercel Inc., "Next.js: The React framework for the web," https://nextjs.org/, 2024, accessed: May 2026.

[13] Google Chrome DevTools Team, "Puppeteer: A Node.js library for controlling headless Chrome," https://pptr.dev/, 2024, accessed: May 2026.

[14] M. Foley and S. Maffeis, "Haxss: Hierarchical reinforcement learning for xss payload generation," in *Proceedings of the 2022 IEEE 21st International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom '22)*, 2022, pp. 147–158.

[15] S. Khan, "Ll-xss: End-to-end generative model-based xss payload creation," in *Proceedings of the 21st International Conference on Learning and Technology (L&T 2024)*, 2024, pp. 121–126.

[16] Y. Frempong, Y. Snyder, E. Al-Hossami, M. Sridhar, and S. Shaikh, "Hijax: Human intent javascript xss generator," in *Proceedings of the 18th International Conference on Security and Cryptography (SECRYPT 2021)*. SciTePress, 2021, pp. 798–805.

[17] B. Pala, L. Pisu, S. L. Sanna, D. Maiorca, and G. Giacinto, "A targeted assessment of cross-site scripting detection tools," in *Proceedings of the Italian Conference on CyberSecurity (ITASEC 2023)*, ser. CEUR-WS.org, vol. 3488. Bari, Italy: CEUR Workshop Proceedings, 2023, pp. 322–334, creative Commons Attribution 4.0 International (CC BY 4.0). [Online]. Available: https://ceur-ws.org/Vol-3488/paper26.pdf

[18] T. Fink, "Automated xss vulnerability detection through context aware fuzzing and dynamic analysis," Diploma Thesis, Technische Universität Wien, 2018. [Online]. Available: https://repositum.tuwien.at/handle/20.500.12708/7741

[19] X. Song, R. Zhang, Q. Dong, and B. Cui, "Grey-box fuzzing based on reinforcement learning for xss vulnerabilities," *Applied Sciences*, vol. 13, no. 4, 2023. [Online]. Available: https://www.mdpi.com/2076-3417/13/4/2482

[20] D. Miczek, D. Gabbireddy, and S. Saha, "Leveraging llm to strengthen ml-based cross-site scripting detection," *arXiv preprint*, 2025, arXiv:2504.21045. [Online]. Available: https://arxiv.org/html/2504.21045v1

Appendix A: Screenshots

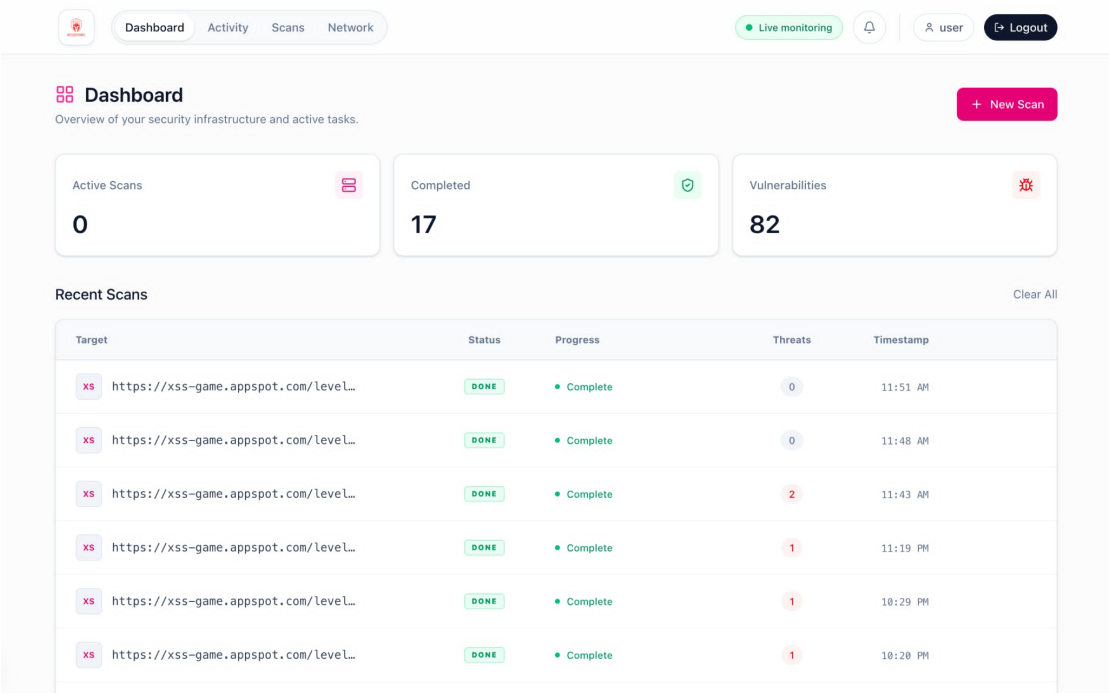

Figure A.1: *RedSentinel Dashboard*

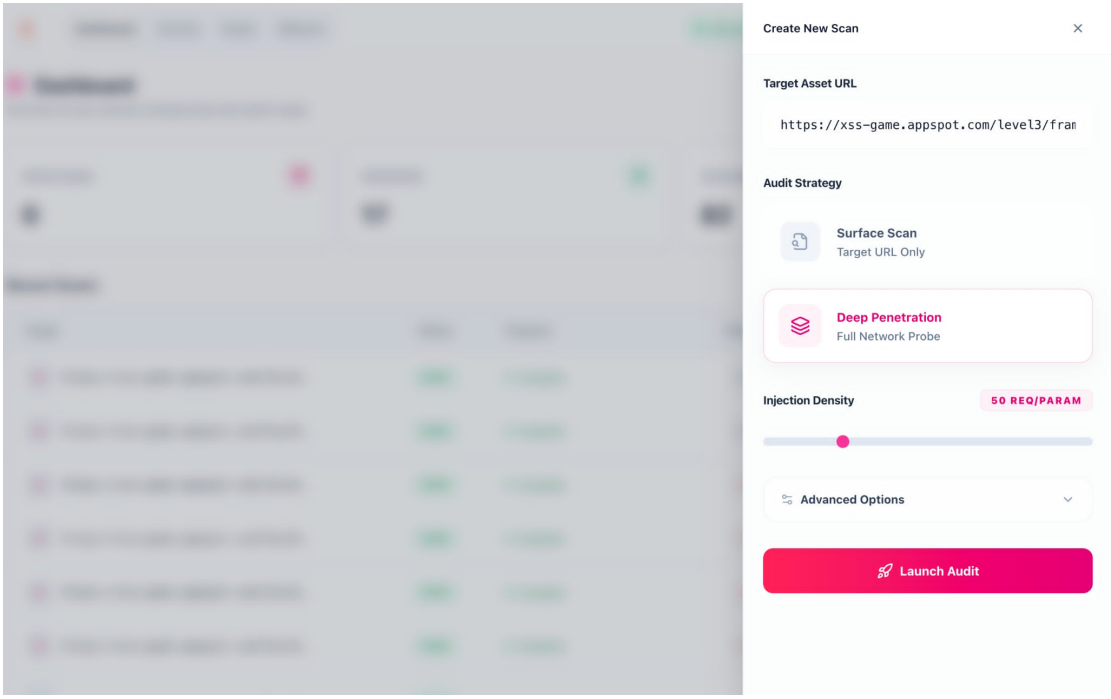

Figure A.2: *New Scan Configuration*

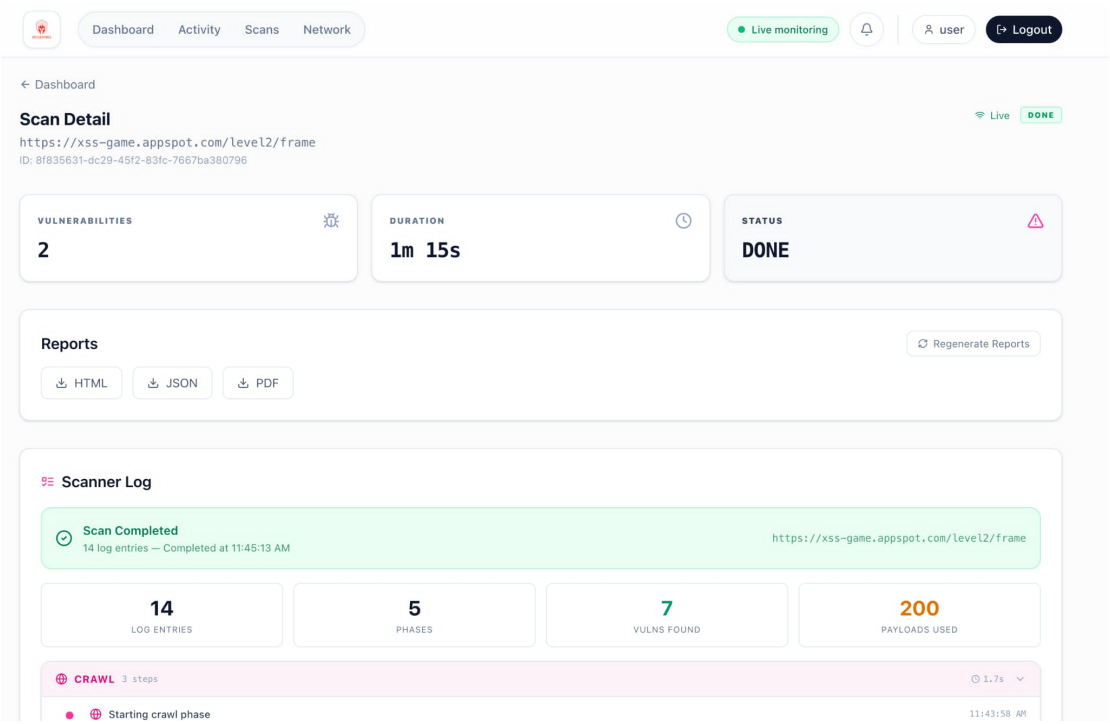

Figure A.3: *Scan result*

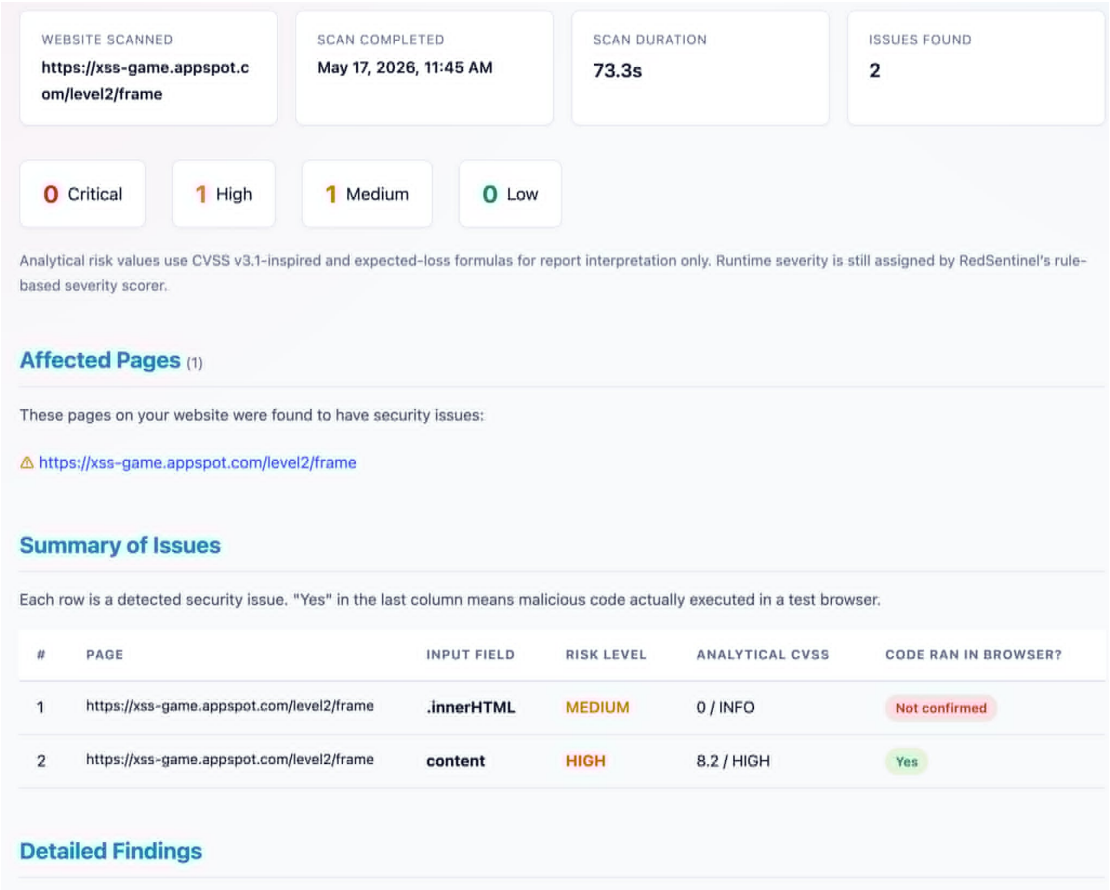

Figure A.4: *Scan summary report for* `xss-game.appspot.com/level2/frame`.

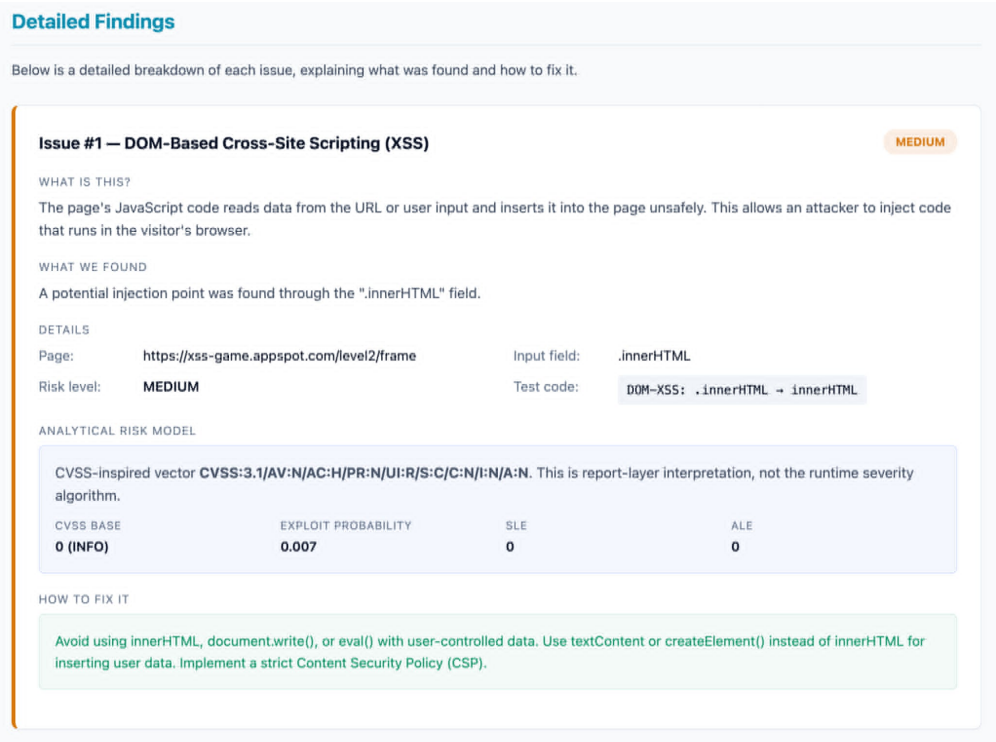

Figure A.5: *Issue #1 — DOM-Based Cross-Site Scripting (XSS) vulnerability report (severity: MEDIUM). A potential injection point was identified via the* `.innerHTML` *field at* `xss-game.appspot.com/level2/frame`.

Figure A.6: *Issue #2 — Stored Cross-Site Scripting (XSS) vulnerability report (severity: HIGH). The* `content` *input field at* `xss-game.appspot.com/level2/frame` *was found to execute injected JavaScript.*

Appendix B: RedSentinel Runtime Data Flow

B.1 Overview

This appendix summarises the runtime data flow of RedSentinel from scan creation to final report generation. The purpose is to show the main data structures exchanged between the dashboard, core API, queue, Python microservices, target application, report engine, and training collector.

```
User Input
  -> Dashboard
  -> Core API
  -> BullMQ Queue
  -> Crawler
  -> Context Module
  -> Payload Generation Module
  -> Fuzzer Module
  -> Target Application
  -> Reflection / DOM / Browser Verification
  -> Severity and Risk Scoring
  -> Report Generation
  -> Runtime Training Data Collection
```

Listing 1: *Overall scan pipeline*

B.2 Dashboard Scan Request

The scan begins when the user submits a target URL and scan options through the dashboard.

```
POST http://localhost:4000/api/scans
Content-Type: application/json

{
  "url": "http://localhost:9090/reflected/body?q=test",
  "options": {
    "singlePage": true,
    "maxParams": 10,
    "verifyExecution": true,
    "maxPayloadsPerParam": 50,
    "timeout": 30000,
    "wafBypass": false
  }
}
```

Listing 2: *Dashboard to Core API request*

```
{
  "id": "62587fda-d7d2-4545-ad2b-321ab30c28a4",
```

```
  "url": "http://localhost:9090/reflected/body?q=test",
  "status": "PENDING",
  "progress": 0,
  "phase": "CRAWL",
  "createdAt": "2026-05-21T10:30:00.000Z"
}
```

Listing 3: *Core API scan creation response*

B.3 Optional Authentication Data

If the target requires authentication, the scan options may include cookie-based or login-based credentials.

```
{
  "auth": {
    "type": "cookie",
    "cookie": "session=abc123def456",
    "username": "admin",
    "password": "admin123"
  }
}
```

Listing 4: *Optional authentication options*

When authentication is not supplied, the scan continues as an unauthenticated scan.

B.4 WebSocket Progress Events

The dashboard subscribes to scan events using the scan ID.

```
{
  "event": "subscribe",
  "data": {
    "scanId": "62587fda-d7d2-4545-ad2b-321ab30c28a4"
  }
}
```

Listing 5: *WebSocket subscription message*

Typical progress events are:

```
{"event":"scan.update","data":{"status":"CRAWLING","progress":5,"
   phase":"CRAWL"}}
{"event":"scan.update","data":{"status":"ANALYZING","progress":25,"
   phase":"CONTEXT"}}
{"event":"scan.update","data":{"status":"GENERATING","progress":50,"
   phase":"PAYLOAD_GEN"}}
{"event":"scan.update","data":{"status":"FUZZING","progress":75,"
   phase":"FUZZ"}}
```

```
{"event":"scan.vuln","data":{"param":"q","type":"reflected_xss","
    severity":"HIGH"}}
{"event":"scan.update","data":{"status":"REPORTING","progress":90,"
    phase":"REPORT"}}
{"event":"scan.done","data":{"status":"DONE","progress":100}}
```

Listing 6: *WebSocket scan event sequence*

B.5 Core Scan Record and Queue Job

The core service stores the scan as a database record.

```
{
  "id": "62587fda-d7d2-4545-ad2b-321ab30c28a4",
  "url": "http://localhost:9090/reflected/body?q=test",
  "status": "PENDING",
  "phase": "CRAWL",
  "progress": 0,
  "options": {
    "singlePage": true,
    "verifyExecution": true,
    "maxPayloadsPerParam": 50
  },
  "createdAt": "2026-05-21T10:30:00.000Z",
  "updatedAt": "2026-05-21T10:30:00.000Z",
  "completedAt": null,
  "error": null
}
```

Listing 7: *Simplified scan database record*

A BullMQ job is then created for asynchronous processing.

```
{
  "name": "scan",
  "jobId": "62587fda-d7d2-4545-ad2b-321ab30c28a4",
  "data": {
    "scanId": "62587fda-d7d2-4545-ad2b-321ab30c28a4"
  },
  "attempts": 3
}
```

Listing 8: *BullMQ scan job payload*

B.6 Crawl Output

The crawler converts the submitted URL into a list of testable endpoints and parameters.

```
[
  {
```

```
    "url": "http://localhost:9090/reflected/body",
    "method": "GET",
    "params": ["q"],
    "source": "url",
    "examples": {
      "q": "test"
    }
  }
]
```

Listing 9: *Crawler output for single-page scan*

For recursive scans, the same format is repeated for all discovered links, forms, and API endpoints.

B.7 Context Module Input and Output

The crawler output is sent to the Context Module for reflection and context analysis.

```
{
  "endpoints": [
    {
      "url": "http://localhost:9090/reflected/body",
      "method": "GET",
      "params": ["q"],
      "examples": {
        "q": "test"
      }
    }
  ]
}
```

Listing 10: *Context Module input*

The Context Module injects a probe marker and records whether the marker is reflected.

```
GET http://localhost:9090/reflected/body?q=rs0xa1b2c3d4
```

Listing 11: *Probe request format*

```
{
  "endpoint": "http://localhost:9090/reflected/body?q=test",
  "params": [
    {
      "name": "q",
      "reflects": true,
      "context": "html_body",
      "context_confidence": 0.97,
      "reflection_position": "html_body",
```

```
      "probe_marker": "rs0xa1b2c3d4",
      "probe_method": "GET",
      "allowed_chars": ["<", ">", "\"", "'", "/", "\\", "(", ")",
   "{", "}", ";", "=", "`", "&", "|"]
    }
  ]
}
```

Listing 12: *Context Module output*

B.8 Payload Generation Input and Output

The context result is sent to the Payload Generation Module.

```
POST http://localhost:5002/generate

{
  "endpoint_url": "http://localhost:9090/reflected/body",
  "context": "html_body",
  "params": {
    "q": {
      "reflects": true,
      "allowed_chars": ["<", ">", "\"", "'", "/", "\\", "(", ")",
   "{", "}", ";", "=", "`", "&", "|"]
    }
  },
  "max_payloads": 50,
  "waf_bypass": false
}
```

Listing 13: *Payload Generation Module input*

The module returns ranked, context-matched payloads.

```
{
  "payloads": [
    {
      "payload": "<img src=x onerror=alert(1)>",
      "target_param": "q",
      "confidence": 0.89,
      "technique": "original",
      "severity": "medium"
    },
    {
      "payload": "<svg onload=alert(1)>",
      "target_param": "q",
      "confidence": 0.87,
      "technique": "original",
```

```
      "severity": "medium"
    },
    {
      "payload": "<script>alert(1)</script>",
      "target_param": "q",
      "confidence": 0.85,
      "technique": "original",
      "severity": "medium"
    }
  ]
}
```

Listing 14: *Payload Generation Module output*

B.9 Fuzzer Module Input

The selected payloads are sent to the Fuzzer Module.

```
POST http://localhost:5003/fuzz

{
  "url": "http://localhost:9090/reflected/body",
  "payloads": [
    {
      "payload": "<img src=x onerror=alert(1)>",
      "target_param": "q",
      "confidence": 0.89,
      "technique": "original",
      "severity": "medium"
    }
  ],
  "verify_execution": true,
  "timeout": 60000,
  "stored_mode": false
}
```

Listing 15: *Fuzzer Module input*

B.10 HTTP Request and Response Data

The fuzzer injects each payload into the target parameter.

```
GET /reflected/body?q=%3Cimg%20src%3Dx%20onerror%3Dalert(1)%3E HTTP
    /1.1
Host: localhost:9090
User-Agent: Mozilla/5.0 (compatible; RedSentinel/1.0)
Accept: text/html
```

Listing 16: *Fuzzer HTTP request*

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<html>
<body>
  <p>You entered: <b><img src=x onerror=alert(1)></b></p>
</body>
</html>
```

Listing 17: *Target HTTP response*

The raw fuzzer result is stored in the following format.

```
{
  "payload": "<img src=x onerror=alert(1)>",
  "target_param": "q",
  "url_sent": "http://localhost:9090/reflected/body?q=%3Cimg%20src%3
   Dx%20onerror%3Dalert(1)%3E",
  "status_code": 200,
  "response_headers": {
    "Content-Type": "text/html; charset=utf-8"
  },
  "response_snippet": "...<b><img src=x onerror=alert(1)></b>...",
  "reflected": false,
  "executed": false,
  "vuln": false,
  "evidence": {}
}
```

Listing 18: *Raw fuzzer result format*

B.11 Reflection Check Output

The reflection checker updates the result after comparing the payload with the HTTP response.

```
{
  "payload": "<img src=x onerror=alert(1)>",
  "reflected": true,
  "exact_match": true,
  "reflection_position": "html_body",
  "reflection_snippet": "...<b><img src=x onerror=alert(1)></b>..."
}
```

Listing 19: *Reflection check output*

B.12 DOM XSS Scan Output

For DOM-based targets, the scanner records source-to-sink evidence.

```
{
  "dom_vuln": true,
  "source": "URLSearchParams.get",
  "sink": "innerHTML",
  "line": 6,
  "snippet": "document.getElementById('output').innerHTML = data;"
}
```

Listing 20: *DOM XSS scan result format*

B.13 Browser Verification Output

If browser verification is enabled, reflected payloads are opened in a headless browser.

```
{
  "payload": "<img src=x onerror=alert(1)>",
  "executed": true,
  "dialog_triggered": true,
  "dialog_type": "alert",
  "dialog_text": "1",
  "console_errors": [],
  "confirmed": true
}
```

Listing 21: *Browser verification output*

A non-executing reflected payload is stored as:

```
{
  "payload": "<script>alert(1)</script>",
  "reflected": true,
  "executed": false,
  "dialog_triggered": false,
  "confirmed": false
}
```

Listing 22: *Negative browser verification output*

B.14 Final Fuzzer Result

The Fuzzer Module returns one result object per payload.

```
{
  "results": [
    {
      "payload": "<img src=x onerror=alert(1)>",
      "target_param": "q",
      "vuln": true,
      "type": "reflected_xss",
```

```
      "reflected": true,
      "exact_match": true,
      "executed": true,
      "dialog_triggered": true,
      "reflection_position": "html_body",
      "technique": "original",
      "severity": "high",
      "evidence": {
        "responseCode": 200,
        "reflectionPosition": "html_body",
        "browserAlertTriggered": true,
        "exactMatch": true,
        "response_snippet": "...<b><img src=x onerror=alert(1)></b
   >..."
      },
      "context_classification": {
        "param": "q",
        "context": "html_body",
        "method": "GET",
        "confidence": 0.97
      }
    }
  ]
}
```

Listing 23: *Final fuzzer result format*

B.15 Severity and Risk Output

The core service calculates severity for each confirmed vulnerability.

```
{
  "severityScore": 80,
  "severity": "HIGH",
  "severityBreakdown": {
    "execution": 40,
    "shareability": 30,
    "sinkDanger": 10,
    "payload": 10
  }
}
```

Listing 24: *Severity score format*

The scan-level risk result is stored as:

```
{
  "riskScore": 85,
```

```
  "riskLevel": "CRITICAL",
  "topVulnType": "reflected_xss",
  "exploitableWithoutAuth": true,
  "browserConfirmedCount": 18,
  "totalVulnCount": 18
}
```

Listing 25: *Risk analysis format*

B.16 Final Report Data Format

The report engine generates JSON, HTML, and PDF reports. The JSON report follows this structure.

```
{
  "scanId": "62587fda-d7d2-4545-ad2b-321ab30c28a4",
  "url": "http://localhost:9090/reflected/body?q=test",
  "status": "DONE",
  "summary": {
    "totalPayloads": 50,
    "reflected": 42,
    "executed": 18,
    "confirmedVulns": 18,
    "severityBreakdown": {
      "CRITICAL": 3,
      "HIGH": 8,
      "MEDIUM": 5,
      "LOW": 2,
      "INFO": 0
    }
  },
  "vulns": [
    {
      "id": "vuln-001",
      "param": "q",
      "payload": "<img src=x onerror=alert(1)>",
      "type": "reflected_xss",
      "severity": "HIGH",
      "url": "http://localhost:9090/reflected/body?q=%3Cimg%20src%3Dx
   %20onerror%3Dalert(1)%3E",
      "reflected": true,
      "executed": true,
      "evidence": {
        "responseCode": 200,
        "reflectionPosition": "html_body",
        "browserAlertTriggered": true,
        "exactMatch": true,
        "severityScore": 80
```

```
      }
    }
  ],
  "riskAnalysis": {
    "riskScore": 85,
    "riskLevel": "CRITICAL"
  },
  "metadata": {
    "target": "http://localhost:9090",
    "targetName": "exploitable",
    "scanDuration": "60.2s",
    "fuzzerVersion": "1.0.0"
  }
}
```

Listing 26: *Final report format*

B.17 Runtime Training Sample Format

After fuzzing, useful runtime results are appended to the ranker-training JSONL file.

```
{
  "timestamp": "2026-05-21T12:00:00.123456",
  "url": "http://localhost:9090/reflected/body",
  "payload_text": "<img src=x onerror=alert(1)>",
  "target_param": "q",
  "context": "html_body",
  "waf": "none",
  "technique": "original",
  "severity": "medium",
  "allowed_chars": ["<", ">", "\"", "'", "/", "\\", "(", ")", "{",
   "}", ";", "=", "`", "&", "|"],
  "response_snippet": "...<b><img src=x onerror=alert(1)></b>...",
  "sink_name": null,
  "source_name": null,
  "dataflow": null,
  "executed": true,
  "dialog_triggered": true,
  "reflected": true,
  "exact_match": true,
  "reflection_position": "html_body",
  "success": true
}
```

Listing 27: *Runtime training sample format*

B.18 Evaluation Result Format

The evaluation pipeline compares expected and detected vulnerabilities.

```
{
  "tp": 14,
  "fn": 1,
  "tn": 1,
  "fp": 0,
  "precision": 1.0,
  "recall": 0.933,
  "f1": 0.965
}
```

Listing 28: *Evaluation metric format*

Evaluation archives follow this structure.

```
eval/archive/2026-05-21_10-30-00/
|-- _meta.json
|-- manifest_frozen.json
|-- results/
|   |-- reflected-body.json
|   |-- reflected-attribute.json
|   |-- dom-innerhtml.json
|   |-- raw_responses/
|       |-- reflected-body.json
|-- summary.json
|-- portswigger.json
```

Listing 29: *Evaluation archive structure*

B.19 Stored XSS Data Flow Format

Stored XSS follows the same pipeline but uses a submit-and-poll structure.

```
[
  {
    "url": "http://localhost:9090/stored/comments",
    "params": ["name", "body"],
    "method": "POST",
    "form": true,
    "stored_endpoint": true
  },
  {
    "url": "http://localhost:9090/stored/comments",
    "params": [],
    "method": "GET",
    "renders_stored_data": true
  }
]
```

Listing 30: *Stored XSS crawler output*

```
{
  "scanId": "scan-002",
  "url": "http://localhost:9090/stored/comments",
  "status": "DONE",
  "summary": {
    "totalPayloads": 3,
    "confirmedVulns": 1,
    "severityBreakdown": {
      "HIGH": 1
    }
  },
  "vulns": [
    {
      "param": "body",
      "payload": "<img src=x onerror=alert(1)>",
      "type": "stored_xss",
      "severity": "HIGH",
      "executed": true,
      "stored": true,
      "evidence": {
        "submitMethod": "POST",
        "pollUrl": "http://localhost:9090/stored/comments",
        "browserAlertTriggered": true
      }
    }
  ]
}
```

Listing 31: *Stored XSS final result format*

B.20 Summary

The runtime flow shows that RedSentinel records each major object passed through the system: user scan options, scan database record, queue job, crawler output, context classification result, generated payloads, fuzzing request, HTTP response evidence, reflection result, DOM result, browser-verification result, severity score, risk score, final report, and runtime training sample.